Промежуточный отчет по диплому: Облачная система управления движением мобильного робота (AGV Denford)

1. Анализ предметной области

Облачные технологии в промышленной автоматизации. Современная промышленная автоматизация все активнее опирается на **технологии облачных вычислений и IIoT (Industrial Internet of Things)**. Объединение оборудования в единую сеть с центральными облачными ресурсами позволяет существенно повысить эффективность и гибкость производства[1][2]. В концепции **«облачной робототехники»** вычислительные мощности, хранилища данных и сервисы искусственного интеллекта выносятся в облако, образуя своего рода *«мозг»* робототехнической системы[2]. Это дает роботам доступ к практически неограниченным ресурсам для обработки данных, совместного обучения и хранения знаний, снижая требования к бортовым вычислениям и стоимости робототехники. Таким образом, возможно создание более «умных» и дешевых роботов с вынесенным в облако интеллектом[2]. Человеческий оператор при этом также может удаленно ставить задачи роботам через сетевые интерфейсы.

Промышленный интернет вещей (IIoT) – фундамент Индустрии 4.0 – предполагает подключение всех устройств (станков, конвейеров, роботов, датчиков) к общей цифровой среде[3][1]. Облачная модель выступает ядром этой среды, позволяя собирать телеметрию, оперативно управлять оборудованием и перераспределять ресурсы между узлами производства[1]. За счет облака достигается интеграция ранее разрозненных систем, их совместная оптимизация и централизованный анализ данных. Например, в облако можно передавать параметры работы роботов, статистику простоев, данные об ошибках – и на основе больших данных улучшать алгоритмы управления.

Мобильные роботы в логистике (AGV/AMR). На складах и производственных линиях все шире применяются автономные транспортные роботы – AGV (Automated Guided Vehicle) и AMR (Autonomous Mobile Robot). **AGV** – это роботы, следующие по заранее заданным маршрутам (например, по нанесенной линии или магнитной ленте) для перевозки грузов[4][5]. Классический пример – логистические тележки, перемещающиеся по складу строго по отмеченному пути и останавливающиеся в заданных точках. **AMR**, в отличие от AGV, способны самостоятельно прокладывать маршрут в динамической среде, используя датчики и карты – т.е. обладают большей автономностью и не требуют фиксированной траектории[6][7]. На практике обе концепции находят применение: AGV эффективны для предсказуемых маршрутов, AMR – для гибких задач.

Примеры внедрения и актуальные технологии. Самый известный пример массового применения складских роботов – компания **Amazon Robotics** (ранее Kiva Systems). Amazon приобрела Kiva и интегрировала тысячи роботов на своих складах, реализуя концепцию *«товар к человеку»*: мобильные платформы (AGV) перевозят стеллажи с

товарами к стационарным сборщикам[8][9]. Роботы Amazon ориентируются по QR-меткам на полу и останавливаются в точках выдачи по команде системы; это позволило **вдвое увеличить емкость склада и ускорить сбор заказов**[10][9]. В России похожие решения разрабатывают компании **ИНТЭК, MetraRobotics**, а холдинг Ростех планирует выпуск тяжелых транспортных роботов грузоподъемностью до 20 тонн; свою модель логистического робота создает и **Яндекс**[11]. Эти примеры показывают, что облачно-управляемые роботы становятся неотъемлемой частью современных производственных и складских систем.

ROS и MQTT – стандартные инструменты. В сфере робототехники де-факто стандартом программной платформы стал **ROS (Robot Operating System)**. ROS представляет собой **экосистему для разработки программ роботов**, обеспечивающую стандартные функции: аппаратную абстракцию, обмен сообщениями между процессами (публикация/подписка), набор готовых библиотек для навигации, управления и др.[12]. Архитектура ROS распределенная: вычисления разбиваются на узлы, обменивающиеся данными в графе сообщений[12]. Это значительно ускоряет разработку за счет повторного использования существующих алгоритмов и модулей, а также облегчает масштабирование на группу роботов[13][14]. Многие алгоритмы SLAM, планирования траектории, управления приводами уже реализованы в пакетах ROS и доступны сообществу. Однако ROS изначально создавался для локальных сетей робота; интеграция его с облаком требует специальных шлюзов или использования ROS 2 с DDS.

Для облачной же IoT-интеграции широко применяется легковесный протокол **MQTT (Message Queuing Telemetry Transport)**. MQTT – протокол прикладного уровня для обмена сообщениями по схеме «издатель – брокер – подписчик», работающий поверх TCP/IP и оптимизированный для устройств с ограниченными ресурсами[15]. Он надежно доставляет сообщения (поддерживает QoS), но при этом накладные расходы значительно ниже, чем у HTTP[16]. Стандарт MQTT предполагает передачу полезных данных в виде компактных JSON-пакетов, что экономит трафик[17]. Благодаря простоте и эффективности MQTT стал **де-факто стандартом в индустриальных IoT-системах**[18][16]. Интересно, что ROS и MQTT концептуально схожи: ROS тоже использует модель паб-саб для своих топиков. Существуют проекты по интеграции ROS с MQTT – через мосты, преобразующие ROS-сообщения в MQTT и обратно[19][20]. Это позволяет, к примеру, отправлять команды из облака через MQTT-брокер на робот, где мост доставит их во внутренняя ROS-сеть[19]. Таким образом, ROS обеспечивает *локальный интеллект* (навигация, обработка сенсоров), а MQTT – *глобальную связность* (облачный контроль, межсистемная коммуникация)[19][20].

Облачные платформы для индустриального IoT. В последние годы появилось множество сервисов, упрощающих создание облачно-робототехнических систем. Сторонние **MQTT-брокеры** и IoT-платформы позволяют не разворачивать собственный сервер, а использовать готовую облачную инфраструктуру. Популярны, например, публичные брокеры **HiveMQ**, **Eclipse Mosquitto**, а также облачные IoT-платформы **Yandex.Cloud (IoT Core)** и **AWS IoT**[21]. Они обеспечивают масштабируемость, безопасность и веб-интерфейсы для мониторинга устройств. Для более сложных задач управления парком роботов существуют решения вроде **AWS IoT Greengrass** –

фреймворк, позволяющий развертывать программные модули на роботах, управлять обновлением ПО «по воздуху» и безопасно связывать роботов с облаком[22][23]. Комбинация AWS Greengrass с ROS 2 демонстрирует, как добавить “умные” облачные функции роботу и облегчить сбор телеметрии, мониторинг состояния парка, удаленное вмешательство оператора[24][22]. Российские аналоги также появляются – например, **Yandex IoT Core** позволяет организовать обмен сообщениями с устройствами через MQTT, интегрировать данные в другие облачные сервисы Яндекса и управлять устройствами через веб-интерфейс[25]. Таким образом, технологий и инструментов для облачного управления роботами предостаточно – выбор зависит от конкретных требований проекта (безопасность, автономность, бюджет). В нашем проекте будут сочетаться некоторые из упомянутых подходов: легкий MQTT для связи и, по возможности, интеграция с существующей облачной платформой.

2. Анализ задачи

Исходное состояние AGV Denford и проблемы

Описание исходной системы. Учебный мобильный робот **Denford AGV** – это автономная тележка на колесном ходу, разработанная в 1990-х для гибких производственных систем. Он представляет собой батарейный напольный транспорт с системой следования по линии. Корпус AGV – алюминиевая платформа на колесах, внутри которой установлены все механизмы[26]. Основой управления служит фирменный контроллер **Denstep**[27]. Контроллер Denstep способен напрямую управлять тремя шаговыми двигателями, имеет аналоговые выходы для регулирования скорости, **16 дискретных входов/выходов**, два аналоговых входа, а также специальные входы концевиков[28]. В конструкции AGV Denford используется один **постоянный мотор с редуктором** на 30:1 для тяги (приводит одно из колес) и **шаговый двигатель для руля**, поворачивающий управляемое колесо через червячную передачу[29]. Такая схема обеспечивает маневренность: реализовано активное управление задним колесом, что позволяет уменьшить радиус поворота и эффективно маневрировать на ограниченном пространстве[30]. Питание осуществляется от двух встроенных 12В аккумуляторов 17А·ч; имеется встроенное зарядное устройство, автоматически отключающееся при полной зарядке[31].

Датчики и органы управления. Робот оборудован множеством сенсоров для автономной работы. Для следования по маршруту используются **оптические датчики линии**: две инфракрасных головки снизу спереди, настроенные на контраст «белая полоса – черный фон»[32]. Белая лента на полу служит путеводящей линией, окаймленной черными полосами по краям; когда левый или правый датчик наезжает на черный цвет, шаговый двигатель поворота отклоняет колесо от черной зоны обратно на белую линию[33]. Таким образом реализовано простое слежение за траекторией без «рыскания». Для автоматических остановок в нужных местах AGV оснащен **ИК-датчиками стыковки** по бокам корпуса[34]. На станциях установлены инфракрасные излучатели; когда приемник робота фиксирует сигнал станции (свет включен) – робот останавливается точно у нее. Одновременно робот своим ИК-передатчиком сообщает станции о прибытии (для координации с хост-системой)[35]. Когда станционный сигнал гаснет (система разрешает движение) – AGV продолжает путь[36]. Помимо этого,

спереди установлено устройство предотвращения столкновений – **сенсор препятствий («радар») Qprox**[37]. При появлении препятствия по ходу движения робот останавливается и не продолжает путь, пока преграда не убрана[38]. Для предупреждения персонала при этом срабатывает **звуковой оповещатель** – AGV воспроизводит записанное голосовое сообщение через динамик на корпусе[38]. На корме робота расположена панель оператора с основными элементами: главной кнопкой-изолятором питания, гнездом подключения зарядного устройства, индикатором зарядки и кнопками ручного управления[39][40]. В частности, на панели имеются кнопки *LEFT/RIGHT* для ручного движения – при нажатии робот может ехать влево или вправо по команде оператора, минуя автоматический режим[40].

Принцип работы в существующей ГПС. Рассматриваемый AGV входит в состав учебной гибкой производственной системы (ГПС) Bauman FMS производства Denford[41]. Он взаимодействует с другими модулями (складом, конвейером, станками) через систему **доковых станций**: на пути проложены несколько станций (стойки) с ИК-маяками. Алгоритм работы исходной системы таков: оператор через ПО на **Host-компьютере** задает команду на перемещение детали, после чего **робокар** (AGV) должен проехать по маршруту (по линии) и остановиться у требуемой станции для загрузки/разгрузки[42]. Denford FMS использует два режима: *автоматический*, когда робокар двигается сам по линии между станциями, и *ручной*, для которого оператор может на самой тележке кнопками двигать ее при настройке[40]. Чтобы запустить робокар в автоматическом режиме, в исходной системе требуется физическая подготовка: **робота ставят на линию**, поворачивают ключ питания (*Battery Isolator*) и нажимают кнопку *RESET* несколько раз, пока контроллер не откалибруется по линии (двойной звуковой сигнал означает, что линия найдена)[43]. После этого AGV автоматически едет до ближайшей станции и останавливается[44]. Дальнейшие перемещения задаются через программу **AGV** на Host-ПК (в составе Cell Software) – фактически, система отправляет роботу команды двигаться к определенной станции, и робокар, считая полоски на линии или реагируя на станции, доезжает до нужной точки и останавливается[45][46]. Если требуется проверить полный круг, оператор может принудительно заставлять робот проехать мимо станции, закрыв ИК-окно на станции (робот воспримет как отсутствие сигнала и поедет дальше)[47]. Таким образом, **исходный AGV Denford функционирует автономно, следуя заложенному алгоритму (следование линии и остановки)**. Взаимодействие с верхним уровнем минимально: Host-компьютер может запускать/останавливать робота через программу (вероятно, по кабельному или радиоканалу связи с Denstep) и обрабатывать сигналы о его прибытии на станции.

Проблемы и ограничения старой системы. Проведенный анализ показал несколько серьезных **недостатков существующей реализации**. Во-первых, **отсутствует полноценное удаленное управление**: система не имеет развитого интерфейса для оператора, все действия запуска/остановки требуют непосредственного присутствия (вставить ключ, нажать reset и т.д.), а сценарии движения зашиты жестко в код[48][43]. Во-вторых, **гибкость крайне низкая** – робот может двигаться только по заранее проложенной линии; изменение маршрутов или логики требует физической перекладки ленты или модификации прошивки Denstep, что делает систему немасштабируемой[48]. Третья проблема – **устаревание аппаратуры и ПО**: контроллер Denstep – закрытая

проприетарная плата 1990-х годов, исходный код недоступен и не расширяем[49]. Это затрудняет интеграцию с современными технологиями: например, невозможно напрямую подключить робот к локальной сети или облаку, так как Denstep не поддерживает стандартные протоколы интернета. Кроме того, возникают **вопросы надежности**: длительная эксплуатация могла привести к износу компонентов, а диагностика осложнена отсутствием телеметрии (нет данных о напряжении батарей, состоянии датчиков в реальном времени в удаленном интерфейсе). Наконец, **отсутствует модуль навигации**: робот не способен отклоняться от линии или объезжать препятствия самостоятельно – он строго следует линии и просто ждет, если путь заблокирован. В условиях развития технологий AMR это существенное ограничение, снижающее исследовательскую ценность системы.

Цели и ограничения проекта модернизации

Цели проекта. Основная цель дипломного проекта – **модернизация AGV Denford для работы под управлением облачной системы**. Это означает перевод робота на современную открытую платформу управления, позволяющую ему подключаться к промышленной сети (или интернету) и получать команды от удаленного сервера/оператора в режиме реального времени[50]. Робот должен стать компонентом IIoT-инфраструктуры лаборатории. В частности, планируется реализовать **дистанционное управление**: оператор через веб-интерфейс или приложение сможет запускать/останавливать движение робота, выбирать режим работы (авто/ручной) и задавать задания (например, поехать в точку X) удаленно[51][52]. Второй ключевой задачей является **расширение функциональности навигации**. После модернизации предполагается возможность свободного движения робота не только по заранее проложенному маршруту, но и по произвольным траекторияй в пределах полигона (реализовать прототип системы навигации)[52][53]. Это подведет базу для внедрения алгоритмов автономной навигации **AMR-типа** – например, следование по виртуальной карте, объезд препятствий, реализация стратегии движения по заданным координатам, а не по линии. Таким образом, модернизированный AGV станет экспериментальной платформой для опробования **алгоритмов интеллектуальной мобильности** (SLAM, планирование маршрута и др.), что придаст работе научную новизну.

Кроме того, целью является **интеграция робота в существующую облачную сеть лаборатории** (или создание новой). Планируется подключить AGV к облачному серверу, где будет вестись сбор телеметрии и координация с другими устройствами. Например, робот сможет передавать в облако информацию о своем состоянии: координаты, уровень заряда, обнаруженные препятствия – чтобы оператор и другие модули системы видели картину в целом. В обратную сторону, облако (или центральный сервер ГПС) сможет посылать роботу задания на транспортировку. Это соответствует концепции единой цифровой фабрики, где все узлы «разговаривают» друг с другом через облачное ПО[1][54].

Ограничения и условия. Проект модернизации должен учитывать ряд ограничений. Во-первых, **аппаратные ограничения**: механическая часть робота, датчики и актуаторы желательны к сохранению. Робот исправен (проверка показала работоспособность всех модулей[55]) и обладает ценной периферией – ее нужно задействовать. Поэтому новое

решение должно уметь считывать сигналы существующих датчиков (линии, стыковки, столкновения) и управлять имеющимися двигателями и индикаторами[56][57]. При этом есть технический нюанс: штатные датчики работают на промышленном уровне 12–24 В, тогда как современные контроллеры (одноплатные компьютеры) оперируют логическими уровнями 3.3 В. Непосредственно соединять их нельзя – потребуется разработать адаптерные схемы для согласования сигналов[58][59]. То же касается управления двигателями: необходимо обеспечить выдачу сигналов 24 В на моторы при управляющем уровне 3.3 В с контроллера[60][61]. Значит, одним из ограничений является **разработка электронного интерфейса (платы расширения)** для стыковки нового контроллера с аппаратурой робота[62][63]. Плата должна предусматривать цепи защиты по питанию (снижение 24В до 5В/3.3В, обратная трансляция для силовых выходов) во избежание выхода из строя нового контроллера[64][65].

Во-вторых, **пространственные ограничения**: внутри корпуса AGV мало свободного места[66]. Новые компоненты должны быть компактными и размещены так, чтобы не мешать обслуживанию и не перегружать питание. Нельзя занимать весь внутренний объем устройства, если в этом нет крайней необходимости[66][67]. Вероятно, плата расширения будет установлена на место старого контроллера Denstep, поэтому её габариты должны быть сопоставимы с двумя платами Denstep (ориентировочно несколько десятков квадратных сантиметров)[56].

Следующее ограничение – **безопасность и совместимость с ГПС**. В старой системе AGV был интегрирован в общую систему аварийного останова (E-Stop) и блокировок. Например, на учебном стенде Denford все устройства (станки, робот, конвейер) подключены к центральному блоку Interlock: при срабатывании аварийного стопа питание снимается со всех исполнительных устройств через специальные линии безопасности[68][69]. Модернизируя робота, мы должны сохранить эту функцию – т.е. новый контроллер должен реагировать на сигнал аварийного отключения (например, размыкание цепи E-Stop) и немедленно останавливать робота. Также при интеграции в облако надо учитывать требования информационной безопасности: если планируется внешний облачный сервер, конфиденциальность данных и управление доступом становятся важными. Возможно, по этой причине в учебном контуре предпочтительно использовать либо локальный (приватный) MQTT-брокер, либо защищенный канал к облаку. В дипломной работе будет сделан акцент на *локальной* облачной сети (например, внутри кафедральной лаборатории), чтобы избежать зависимости от интернета и исключить утечки данных[21][70]. Но архитектура должна допускать масштабирование до публичного облака при необходимости.

Наконец, **временные и ресурсные ограничения**: проект – учебно-исследовательский, время ограничено одним учебным годом. Поэтому реализация будет поэтапной. На данном, промежуточном этапе (НИРС) фокус на анализе, концепции и макетировании ключевых компонентов. Полный функционал (например, сложная навигация) может выходить за рамки текущего этапа, но должен быть заложен фундамент под эти возможности. Главными приоритетами являются: получение работающего канала связи робот–облако–оператор, замена контроллера и восстановление базовой движимости робота под новым управлением. Более продвинутые функции (визуальная ориентация,

групповое взаимодействие роботов и т.п.) могут рассматриваться как перспективы последующих исследований.

3. Концепция и возможные пути решения

Для достижения поставленных целей рассматривается несколько концептуальных подходов к архитектуре системы. Ниже проведено сравнение трех основных вариантов: (1) **IoT-подход на базе MQTT + Python**, (2) **робототехнический подход на базе ROS + веб-интерфейс**, и (3) **гибридное решение**, комбинирующее элементы первых двух. Каждый из вариантов оценивается с точки зрения сложности внедрения, требуемых ресурсов и соответствия требованиям.

Вариант 1: MQTT + Python (облачный IoT-подход)

Суть подхода: оснастить робота одноплатным мини-компьютером (или микроконтроллером) с прошивкой, которая напрямую управляет двигателями и датчиками, а связь с оператором организовать через **MQTT-брокер**. Команды от оператора публикуются в MQTT-топиках, на которые подписан бортовой контроллер робота; телеметрия от робота публикуется в другие топики для потребления операторским интерфейсом. Этот вариант фактически реализует концепцию IIoT: робот – «вещь», подключенная к облаку, обменивается сообщениями через брокер. В качестве контроллера можно использовать, например, **Raspberry Pi** под управлением Python-скрипта или даже Wi-Fi микроконтроллер **ESP32** под MicroPython. Облачный брокер – локально развернутый Mosquitto либо сервис типа Yandex IoT Core.

Применение на практике: Данный подход уже опробован в предшествующих работах. Например, в дипломном проекте В.А. Лукоянова (2025) модернизировался аналогичный AGV: вместо Denstep установлен Raspberry Pi с Python-программой, а обмен командами реализован по MQTT[57][50]. Робот получает управляющие сообщения по сети (Wi-Fi) и отправляет обратно статусы – для этого использована библиотека Paho MQTT и свой MQTT-брокер[71][72]. В другом проекте (И.М. Половников, 2025) выбран еще более минималистичный путь: **ESP32** в роли контроллера промышленного робота, работающий под **MicroPython**, связанный с облачным MQTT-брокером **Clusterfly**; операторские команды отправлялись через мобильное приложение IoT MQTT Panel[73][74]. Эти примеры подтверждают, что MQTT+Python подход **вполне жизнеспособен**: даже маломощный микроконтроллер способен обработать MQTT-сообщения и управлять робототехникой, а одноплатный компьютер – тем более.

Преимущества:
– *Простота и легкость*. MQTT очень прост в реализации и требует минимум накладных ресурсов[75][76]. Python – высокоуровневый язык, позволяющий быстро разработать логику управления. В итоге порог входа и скорость разработки высоки: не нужно осваивать сложный ROS, писать драйверы – достаточно использовать готовые MQTT-библиотеки (paho-mqtt и др.).
– *Низкие требования к аппаратуре*. В отличие от ROS, который лучше запускать на полноценной ОС (Linux) и приличном CPU/RAM, MQTT-решение можно внедрить даже на микроконтроллере. Это экономично и энергоэффективно. Контроллер ESP32 (~160 МГц, 520 КБ RAM) стоит в разы дешевле Raspberry Pi, но способен поддерживать MQTT-брокер

и клиенты[77]. При необходимости брокер можно разместить прямо на микроконтроллере (есть реализации MQTT-серверов под ESP32) для полной автономности локальной сети[70][78].
– *Хорошая интеграция с облаком.* MQTT создан для облака, поэтому подключение к любому IoT-сервису (AWS, Yandex, etc.) не требует адаптации – достаточно настроить URL брокера и креденшелы. Кроме того, MQTT-сообщения легко пересылать через интернет, они мало весят и могут шифроваться (TLS), что упрощает удаленный доступ. Плюс MQTT отлично подходит для сбора телеметрии: можно настроить несколько топиков (например, `robot/battery`, `robot/position`, `robot/errors`) и получать на сервер потоки данных для анализа.

– *Гибкость развития.* Система на MQTT – модульная по определению. Легко добавить новые узлы (например, еще одного робота или датчик) – достаточно подписаться на соответствующие топики. Можно подключить стандартные **панели мониторинга**: существуют готовые IoT-клиенты и дашборды (Node-RED, ThingsBoard и пр.), которые интегрируются через MQTT. Это дает потенциал быстро расширять функциональность без перепрошивки каждого устройства.

Недостатки:
– *Ограниченная «интеллектуальность».* Используя чисто MQTT+Python, мы получаем простую систему команд и статусов, но **сложные функции (навигация, картография)** придется реализовывать вручную. В ROS многие алгоритмы доступны «из коробки», а здесь – нужно либо писать самому, либо пытаться подключать сторонние библиотеки. Например, реализовать SLAM на чистом Python с нуля крайне трудоемко. Есть, правда, путь: совмещать Python-код с библиотеками OpenCV, TensorFlow и др. для продвинутых задач, но это усложняет систему и требует мощного компьютера на борту.
– *Ограничения производительности и реального времени.* Микрокомпьютер типа Raspberry Pi способен управлять реальным роботом, однако Linux – не операционная система реального времени, а Python – интерпретируемый язык. Возможны задержки в циклах управления, джиттер при обмене по Wi-Fi. Для несложных задач (движение с небольшой скоростью, периодическое получение команд) это некритично, но если потребуются жесткие временные рамки (например, стабилизация по датчикам десятки раз в секунду), MQTT+Python может не справиться. Впрочем, наш AGV движется медленно, и десятки миллисекунд задержки не страшны.
– *Отсутствие стандартизации в робототехнике.* Каждый компонент нужно проектировать самостоятельно: как код будет структурирован, какие топики использовать, как форматы команд определить – все на усмотрение разработчика. Это дает свободу, но и создает риски «велосипедостроения». В то время как ROS диктует определенную структуру (узлы, сообщения, сервисы) и тем самым упрощает интеграцию разных частей, MQTT-подход требует четко продумать архитектуру взаимодействия, чтобы не возник хаос из разрозненных сообщений. Нужно будет разработать **протокол команд и отчетов** для робота (например, топик `robot/move` с полезной нагрузкой JSON: `{"dir": "forward", "speed": 50}`), и следить за его актуальностью в обеих программах (робот и клиент).

– *Меньше готовых решений для робототехники.* Придется самому писать, например, обработку датчиков линии и принятие решения по коррекции траектории. С нуля –

возможно, но менее эффективно, чем использовать готовые наработки ROS Navigation Stack. Таким образом, данный вариант проще на старте, но **масштабирование функционала потребует значительных усилий**, что может превратить проект в чисто инженерный, отвлекая от научной составляющей (исследования новых алгоритмов).

Вариант 2: ROS + веб-интерфейс

Суть подхода: установить на робота полноценную ОС (например, Ubuntu Mate) и запустить **ROS** (Robot Operating System), развернув набор узлов для управления движением, обработки сенсоров и связи. Для удаленного доступа разработать **веб-интерфейс** или приложение, которое будет взаимодействовать с ROS через специальные мосты (rosbridge, REST API, либо прямое подключение к ROS-master, если сеть общая). То есть облако в данном случае может выступать не столько управляющим звеном, сколько отображающим и командным: основная «логика» работы остается на борту робота (в ROS), а оператор через интернет посылает задания и наблюдает за состоянием. Либо можно на удаленном сервере поднять ROS-мастер и связывать робота с сервером по сети, реализуя распределенную ROS-систему.

Обоснование: Такой подход более традиционен для робототехники. ROS предоставляет все необходимые **сервисы нижнего уровня**: управление моторами через сообщения, чтение датчиков, построение карт, планирование траектории, контроль одометрии, и т.д.[12]. Можно использовать, например, пакет **rosserial** для связи микроконтроллера (если оставить Denstep или Arduino) с ROS-компьютером, либо подключить драйверы двигателей напрямую к Raspberry Pi и использовать GPIO через ROS-пакеты. Для нашего AGV особенно привлекателен ROS тем, что в будущем позволит подключить модули локализации (SLAM) и навигации с минимальными усилиями, воспользовавшись готовыми алгоритмами. Например, если оснастить робота лидаром или камерой, можно задействовать пакет gmapping или cartographer для построения карты, а Move_Base для прокладки маршрута. Таким образом, вариант 2 ориентирован на **долгосрочные научные задачи** – превратить AGV Denford из примитивного следователя линии в **полноценного автономного мобильного робота** с возможностью ориентирования в пространстве.

Реализуемость на примере: Опыт внедрения ROS для мобильных платформ уже есть в нашей лаборатории. В работе Е.О. Ольхова (2022) для задачи ориентирования *робокара* с камерой использовался ROS Kinetic с написанием узлов на Python, реализующих фильтр частиц для локализации[79][80]. ROS позволила Ольхову интегрировать камеру, одометрический датчик и алгоритм SLAM в единую систему, а также тестировать все в симуляции. В нашем случае, хотя задача несколько иная, ROS также окажется полезной: например, можно подключить пакет **ROS-Industrial** для взаимодействия с внешними устройствами FMS, или использовать **ROSbridge** для связи с веб-клиентом. Существует и обратный пример – работа Л.С. Луна *и др.* (2020) демонстрирует интеграцию ROS-ориентированного промышленного робота с облаком AWS: они использовали MQTT-брокер и **ROS-MQTT Bridge**, передающий команды с облачного MQTT в ROS-топики на роботе[19][20]. Это подтверждает, что ROS совместим с облачным управлением: мы могли бы, например, поднять ROS Core на сервере и связать его с ROS на роботе через MQTT-шлюз или VPN-соединение, добившись глобального охвата.

Преимущества:
– *Богатый функционал и научная ценность.* Использование ROS сразу выводит проект на другой уровень по научной значимости. Можно применять продвинутые алгоритмы навигации, компьютерного зрения, координации нескольких роботов – все это уже есть в виде ROS-пакетов. Проект из “инженерной доработки” превращается в **исследовательскую работу**: например, можно экспериментировать с разными методами локализации (AMCL, SLAM), сравнивать их эффективность, оптимизировать параметры – тем самым вносить научный вклад. К ROS легко подключить алгоритмы машинного обучения (через TensorFlow, Pytorch – имеются обертки) для распознавания объектов, если это потребуется. Кроме того, **сообщество ROS огромное**: любой возникающий вопрос (как управлять моторами, как собрать URDF-модель робота) имеет готовые решения или документацию. Это снижает риски и экономит время на разработку базовой функциональности, позволяя сосредоточиться на новизне.

– *Модульность и масштабируемость.* ROS изначально распределен: можно **разделять вычисления** между устройствами. Например, тяжелую задачу картографии выполнять на мощном облачном сервере, а легкие задачи (PID-регуляция моторов) – на борту. ROS2 даже поддерживает прямое взаимодействие по сети без единого мастера, что упрощает распределение. Таким образом, подход ROS + облако масштабируется на несколько роботов (координация через общие топики) и на более сложные системы. Если в будущем в лаборатории появятся новые AGV или манипуляторы, они могут легко интегрироваться в общий ROS-контур.

– *Стандартные интерфейсы.* ROS предоставляет стандартные сообщения для распространенных задач. Например, геометрия траектории задается сообщением типа `geometry_msgs/Twist` (линейная и угловая скорость); лазерные сканы – `sensor_msgs/LaserScan`. Благодаря этому, **разные компоненты совместимы из коробки** – можно взять готовый узел телеприсутствия (joy_teleop) и он сразу сможет управлять нашим роботом, если мы примем стандарт `cmd_vel`. Это уменьшает объем собственного кода и облегчает поддержку.

– *Визуализация и отладка.* В ROS есть мощные инструменты визуализации (RViz), записи данных (rosbag) и мониторинга системы (rqt панели). Для дипломной работы это полезно: например, можно записать данные с датчиков и воспроизвести для анализа, визуализировать траекторию движения робота в реальном времени, построить графики зависимостей. Это повысит научную ценность – мы сможем не только заявить о работоспособности решения, но и **представить количественные результаты** (например, точность позиционирования, задержки, ресурсопотребление).

Недостатки:
– *Сложность и трудоемкость.* Внедрение ROS на реальный прототип – нетривиальная задача. Необходимо знание экосистемы ROS, настройка рабочего окружения (на RPi придется ставить облегченный ROS дистрибутив, оптимизировать работу из-за ограниченной мощности). Пороги вхождения и отладки значительно выше, чем в варианте с простым MQTT. Фактически, придется написать несколько ROS-узлов (для моторов, для сенсоров, для связи) и обеспечить их взаимодействие. Настройка может отнять много времени (например, калибровка параметров локализации, настройка

конфигурационных файлов). Учитывая ограниченное время НИРС, это риск: есть вероятность потратить много усилий на «борьбу с ROS» в ущерб собственно целям проекта.

– *Требования к оборудованию.* ROS1 лучше всего работает на ПК; Raspberry Pi 3 или 4 может его запустить, но ресурсы будут на пределе, особенно если использовать rviz или тяжелые алгоритмы. Это накладывает ограничение: либо требуется более мощный одноплатный компьютер (например, NVidia Jetson, Udoo X86 и т.п.), что увеличивает бюджет, либо мириться с урезанной функциональностью (без графического интерфейса, с ограничением по частоте алгоритмов). Кроме того, **энергопотребление** RPi выше, чем у микроконтроллера – придется тщательно проверить, выдержит ли штатная батарея дополнительную нагрузку (RPi ~5 В 2–3 А). Возможно, потребуется добавить преобразователь питания (что в нашем случае и так планируется) и учитывать сокращение времени автономной работы.

– *Сложность облачной интеграции.* Связать ROS с внешним веб-интерфейсом сложнее, чем MQTT-вариант. Нужно либо запускать rosbridge-сервер на роботе и писать JavaScript-клиент (ROSLibJS) для веб-страницы – это отдельная задача со своими нюансами (безопасность, производительность), либо делать промежуточный сервер, преобразующий команды веба в ROS-сообщения. Существуют решения (например, **ROSbridge** по WebSocket), но их надо изучить, настроить. Альтернативный путь – ROS2, который мог бы общаться с облаком по DDS или прямо по MQTT (в ROS2 есть и такие эксперименты). Но переход на ROS2 еще более усложнит проект.

– *Избыточность для простых задач.* Если наша ближайшая цель – просто **дистанционно управлять роботом и получить от него базовые данные**, ROS может оказаться слишком тяжелым решением. Затраты на внедрение могут не окупиться, если в итоге робот все равно ездит по линии и лишь принимает команды старт/стоп. Проще говоря, если не реализовать полноценные алгоритмы автономии, использование ROS будет неоправданным усложнением системы «ради науки». Комиссия может спросить: зачем был нужен ROS, если можно было все сделать на MQTT? Поэтому необходимо, выбирая ROS, быть готовым продемонстрировать **дополнительные возможности** (например, карту склада, построенную роботом, или работу в симуляторе Gazebo). Это увеличивает объем работ.

Вариант 3: Гибридное решение (ROS + IoT)

Идея: совместить плюсы обоих подходов, чтобы компенсировать их минусы. То есть организовать систему многоуровневой: **на уровне робота – ROS** для локального управления и навигации, **на уровне облака – MQTT/веб** для координации, интерфейса и интеграции с внешними системами. Такой гибрид позволяет использовать ROS там, где он наиболее эффективен (внутри робота для реального времени и алгоритмов), а MQTT – там, где он уместен (между роботами и облаком, для обмена простыми сообщениями и удаленного доступа). Практически это может выглядеть так: на роботе работает ROS-нода, отвечающая за движение, подписанная на локальный топик `cmd_vel`. Робот подключен к MQTT-брокеру мостом, который переводит сообщения из облака (например, топик `robot1/cmd_vel` на брокере) в ROS-топик `cmd_vel` на борту[19][20]. Аналогично, телеметрия: ROS-нода публикует `odom` (одометрия) локально, мост берет эти

данные и отправляет в MQTT топик `robot1/odom` для удаленного мониторинга. Операторский веб-интерфейс работает через MQTT (или HTTP API к брокеру) – т.е. видит данные `robot1/odom` и посылает команды, публикуя `robot1/cmd_vel`. При таком подходе **облачный сервер** может быть реализован на IoT-платформе (типа Yandex IoT Core, AWS IoT) или на собственном брокере; к нему могут быть подключены и другие устройства.

Пример реализации: описанная схема соответствует архитектуре работы, представленной в статье L.S. Luna et al. (2020), где манипулятор KUKA управлялся через ROS, а глобальное наблюдение шло через AWS с MQTT[19][20]. Они сделали мост: MQTT-брокер на AWS EC2 связан с ROS Core на роботе, команды с облака транслируются как ROS-команды манипулятору, а телеметрия – обратно в облако[19][20]. В результате оператор через веб-приложение мог управлять роботом, а на самом роботе ROS-алгоритмы исполняли команды и контролировали двигатели. Аналогично, в проекте R.A. Толмачева (2023) по облачному управлению ГПМ был использован сервис **Yandex IoT Core** в качестве посредника между устройствами (робот, станок) и клиентским приложением. На устройствах (роботе и контроллере станка) работало ПО, отправлявшее и получавшее сообщения из облака, а логика автоматического цикла была заложена на самом устройстве[81][25]. Толмачев добился того, что гибкий модуль мог выполняться полностью автоматически по сценарию, а сценарий (последовательность команд) хранился в облаке и мог редактироваться без перепрошивки устройства[82][83]. Это пример комбинирования: *облако* отвечает за конфигурацию и высокоуровневое расписание, *робот на месте* – за непосредственное управление по заданному сценарию.

Преимущества гибрида:
– *Баланс нагрузки*. Критичные по времени операции (маневрирование, предотвращение столкновений) выполняются локально на роботе, без задержек сети, тогда как тяжелые вычисления или координация – на сервере. Например, робот может локально строить карту помещения, но отправлять ее периодически в облако для объединения с картами от других роботов (мульти-робот SLAM). Или облако может считать оптимальное распределение задач между несколькими AGV, а каждый робот локально реализует свою часть маршрута. Подобное разделение соответствует принципам **Edge Computing** – часть вычислений на “краю” (робот), часть в центре (облако)[2]. Это повышает отказоустойчивость: при потере связи робот не станет беспомощным – он продолжит выполнять локальную задачу (например, доедет до безопасного места), а облако просто временно не будет получать данные.

– *Использование преимуществ ROS и MQTT одновременно*. Робот может использовать готовые ROS-пакеты навигации, а облако – стандартные MQTT-инструменты для мониторинга (например, подключить панель Grafana через MQTT-датасорс для визуализации телеметрии). В итоге система получается и **функционально богатой**, и **удобной для удаленного доступа**. Также, если на роботе ROS, а в облаке MQTT, их легко связать: существуют открытые реализации мостов ROS<–>MQTT (например, пакет rosbridge_mqtt или собственная нода-переводчик)[20][84]. Значит, интеграционные риски снижены – не придется изобретать свой протокол, можно взять наработки сообщества.

– *Гибкость конфигурации облака*. По сравнению с полным ROS-вариантом, где потребовалось бы на сервере поднимать ROS-master (сложнее, требует совместимости версий, сетевых настроек), MQTT-брокер в облаке гораздо проще и **масштабируется** автоматически под нагрузку (в случае использования управляемого сервиса). Мы можем начать, например, с локального Mosquitto на Raspberry Pi в лаборатории, а затем перейти на промышленный брокер (HiveMQ Cloud или Yandex IoT) без изменения логики работы робота – достаточно переписать URL брокера и задать ключи доступа[21][70]. Это важное достоинство: система, созданная на этапе НИРС, может затем быть **масштабирована для промышленного использования** с минимумом доработок.

Недостатки гибрида:
– *Сложность реализации (суммируется). Гибрид требует разбираться и с ROS, и с MQTT, и с тем, как они стыкуются. Нужно обеспечить надежную синхронизацию между двумя мирами, чтобы не возникло ситуации «два капитана в одном корабле». Например, если ROS на роботе самостоятельно решит объехать препятствие, а облако в этот момент посылает команду ехать прямо – должна быть продумана приоритетность** команд. Необходимо разработать протокол взаимодействия: какие действия робот делает автономно, а за чем всегда обращается к облаку. Если этого четко не определить, система может вести себя непредсказуемо. Проектирование такого распределенного алгоритма – нетривиальная научно-инженерная задача.

– *Ресурсы и задержки*. Несмотря на баланс, все равно добавляется *накладные расходы* на коммуникацию между ROS и MQTT. Мост, конвертирующий сообщения, вносит пусть небольшую, но задержку. В критических сценариях (например, аварийный стоп) это может быть проблемой. Правда, E-Stop можно реализовать аппаратно (прервать питание двигателя), тогда задержки ПО не играют роли. Также требуется достаточный сетевой канал: если видеопоток с камеры решим передавать, MQTT не подойдет (нужно RTSP или подобное), а ROS по сети – тоже тяжело без оптимизации. В гибриде, вероятно, придется ограничиться отправкой “легких” данных в облако (координаты, состояние, редкие снимки), то есть **не весь потенциал облака будет использован** из-за ограничений канала.

– *Увеличение количества компонентов*. Система получается более сложной структурно: вместо двух программ (робот и оператор) у нас теперь целый набор – ROS-ноды, брокер, мост, веб-клиент, возможно, отдельный сервер для логики. Это повышает риск сбоев. Необходимо будет тщательно протестировать все связи. Документирование и отладка гибридной системы сложнее, но это окупится надежностью и расширяемостью.

Вывод по концепциям: Сводную характеристику подходов можно представить так:

- **MQTT+Python:** Минимальная сложность внедрения, быстрое получение результата (робот управляется дистанционно), но ограниченные возможности для развития; подходит для демонстрации базового облачного управления и сбора данных[75][76].
- **ROS+Web:** Максимум возможностей для автономности и дальнейших исследований, но высокий порог реализации, риски не уложиться в сроки;

оправдан, если требуется научная новизна в части алгоритмов и планируется серьезная работа над автономной навигацией[12][13].

- **Гибрид:** Компромиссный вариант, более сложный, чем MQTT, но позволяющий добиться и удаленного управления, и использовать ROS-алгоритмы локально; оптимален, если цель – сделать опытный образец для долгосрочных экспериментов, хотя на этапе НИРС может быть сложен.

В рамках данного проекта целесообразно ориентироваться на **последовательный переход**: начать с простого MQTT-варианта, отработать связность и базовые функции, затем постепенно внедрять ROS-компоненты для повышения автономности. Такой путь выбрали, например, коллеги в смежных работах: сначала наладили облачную связь (Толмачев настроил Yandex IoT Core и передачу команд), затем добавили автоматический режим по сценарию на устройстве[83][25]. Мы планируем аналогично: **на промежуточном этапе** реализовать MQTT-управление AGV с удаленного клиента, а **на завершающем этапе ВКР** – интегрировать ROS-модуль навигации (возможно, портировать алгоритмы из работы Ольхова по визуальной локализации[79]). Это позволит постепенно усложнять систему, проверяя работоспособность на каждом шаге.

4. Требования к системе управления

На основе анализа задачи и концепций сформулированы требования к разрабатываемой системе. Они разделяются на **функциональные**, **нефункциональные** (включая архитектурные) и **ограничения** (частично уже упомянутые во введении). Требования соответствуют целям проекта и устраняют недостатки исходной системы[48][85].

Функциональные требования

- **Удаленное управление движением:** Система должна обеспечивать дистанционную отправку команд роботу и выполнение их в реальном времени. Оператор через клиентское приложение должен иметь возможность *запустить/остановить* движение AGV, переключать режимы (автономный/ручной), задавать целевую станцию или маршрут. Робот обязан корректно выполнять полученные команды – например, при команде “manual forward” двигаться вперед, при команде “go to Station 2” – следовать до станции 2 и остановиться.
- **Автономное движение по маршруту:** В автоматическом режиме система должна уметь вести робота по заранее заданному маршруту *без вмешательства оператора*. Маршрут может быть выражен в виде последовательности точек (станций) или как виртуальная карта. На текущем этапе достаточно повторения функционала линии: робот автоматически едет вдоль линии и останавливается на конечной точке, но в перспективе требуется поддержка произвольных маршрутов (то есть система навигации).
- **Обработка датчиков и ситуационная реакция:** Система должна читать показания всех ключевых датчиков AGV и реагировать на них согласно логике безопасности. Например, если сработал **датчик препятствия**, робот обязан немедленно остановиться (даже без команды оператора)[38]. Если пришел

сигнал от станции (докинг-сенсор) – в автоматическом режиме робот должен остановиться для ожидания команды продолжения[34]. При ручном управлении с панели оператора на роботе, команды с облака могут игнорироваться (или наоборот, локальная панель блокируется в удаленном режиме – этот момент надо определить). Также, если уровень батареи упадет ниже порога, система должна выдать предупреждение и при критическом уровне – остановить движение для предотвращения разряда (добавляется датчик напряжения).

- **Передача телеметрии и состояния:** Разрабатываемый программно-аппаратный комплекс должен отправлять в облако (на пульт оператора) **информационные сообщения о своем состоянии**[86][87]. В минимальном варианте это уведомления о выполнении команд и обнаружении нештатных ситуаций (например: “Команда выполнена: достигнута станция 2”, “Обнаружено препятствие, движение приостановлено”). Желательно также передавать текущие параметры: скорость, координаты (если есть), уровень заряда батареи, статус соединения. Операторский интерфейс должен отображать эту телеметрию в удобном виде (панель или консоль). Такие сообщения позволят оператору и системе в целом принимать решения и анализировать эффективность (например, зная времена прохождения участков, можно оптимизировать движение).
- **Интерактивный режим программирования сценария:** (желательное требование) Система должна поддерживать **настройку маршрутов и задач без перепрограммирования контроллера**. Иными словами, менять последовательность действий AGV следует за счет внешних сценариев или параметров. Это может быть реализовано через конфигурационный файл или через облачный интерфейс. Например, оператор может задать сценарий: “По прибытии на станцию 3 подать звуковой сигнал, затем ехать на станцию 1”. В идеале, сценарий загружается в облаке и отправляется роботу, а тот выполняет автоматически[82]. Данное требование повышает гибкость (система **поддерживает изменение сценария работы без изменения кода**[82]), что важно для экспериментальной установки.
- **Совместимость с режимом ГПС:** Поскольку робот входит в комплекс ГПС с другими модулями, система должна уметь работать совместно с ними. Например, в сценарии общей автоматической работы робот должен ждать сигналов от станков/склада через центральный контроллер. Требование: реализовать возможность приема команд не только от человека, но и от внешней **супервизор-системы** (например, от Host-компьютера FMS). Это может быть MQTT-клиент на Host-ПК, публикующий задания роботу на основании состояния других модулей. Таким образом, облачная система управления должна быть **интегрируемой**: открытый интерфейс (API) для внешних приложений.

Нефункциональные требования

- **Надежность и отказоустойчивость:** Система должна работать устойчиво в условиях промышленной среды. Требуется обеспечить стабильное сетевое подключение робота и сервера[85]. При временной потере связи робот должен безопасно останавливаться и не совершать опасных действий. Аппаратная часть спроектирована с запасом по нагрузке и защите: например, учитываются токи и

напряжения всех узлов, чтобы не перегрузить контроллер или драйверы[64][65]. В софт заложены механизмы обработки ошибок: повтор попытки соединения, контроль целостности данных. QoS MQTT-сообщений выбирается такой, чтобы важные команды доходили гарантированно (например, QoS=1 для команд движения)[88]. Кроме того, критические функции (как E-Stop) дублируются аппаратно для надежности.

- **Безопасность управления:** Неавторизованные пользователи не должны получить контроль над роботом. Если используется публичный облачный брокер, то связь шифруется (SSL) и защищается аутентификацией (логин/пароль или сертификаты). Планируется ограничить круг допустимых команд на стороне робота – например, игнорировать любые незнакомые сообщения или явно некорректные команды (во избежание sabotage). В лабораторных условиях, возможно, достаточно закрытой локальной сети Wi-Fi. Вдобавок, система должна соблюдать **технику безопасности**: максимальная скорость движения и ускорения устанавливаются программно, чтобы робот не вышел из контролируемой зоны или не опрокинул груз. Любая команда, противоречащая требованиям безопасности (например, движение, если разряжена батарея, или повторный запуск, если уже авария) – блокируется.
- **Масштабируемость и расширяемость:** Архитектура проектируется модульной и расширяемой. Требование: **поддержка подключения новых устройств** (в будущем) без кардинальной переработки системы. Например, если добавить второй AGV, он должен легко вписаться – для этого адресация команд и топиков делается с учетом идентификатора робота (мы сразу проектируем, например, топики вида `agv1/…` и `agv2/…`). Также, если решим подключить больше датчиков к роботу (камера, лидар), система должна их принять: программный дизайн предусматривать добавление модулей обработки сенсоров. Нефункционально это выражается в принципе **Low Coupling, High Cohesion** – минимальная зависимость между модулями. Например, можно выделить независимые блоки: “блок управления движением”, “блок связи”, “блок навигации” – они взаимодействуют через четко определенные интерфейсы (топики или функции). Тогда замену или модернизацию одного блока (скажем, внедрение нового алгоритма навигации) можно сделать без переписывания остальных.
- **Современность технологий:** Проектируемая система должна **использовать актуальные версии ПО и оборудования**, соответствующие современному уровню развития[89]. Это требование обеспечит ей больший жизненный цикл и научную ценность. На практике это значит: выбираем современные микроконтроллеры и компоненты (например, 32-разрядные MCU вместо устаревших 8-разрядных), пишем код на современных версиях языков (Python 3.x,最新 C++ стандарта для ROS)[89]. Также следование передовым практикам: например, внедрение контейнеризации (Docker) для разворачивания ПО, использование системы контроля версий, соответствие стандартам оформления кода. Такой подход повысит качество и актуальность разработки, что важно для диплома.
- **Удобство взаимодействия с оператором:** Интуитивно понятный **человеко-машинный интерфейс** – одно из требований. Операторское приложение (будь то

веб-страница или настольное приложение) должно наглядно отображать все необходимые данные: состояние робота, заряд, режим, полученные команды и пр.[90]. Управление должно осуществляться простыми понятными элементами (кнопки, переключатели режимов)[91]. Например, можно реализовать на веб-дэшборде: кнопки “Вперед/Назад/Влево/Вправо” для ручного режима, индикаторы подключения к брокеру и текущего топика, окно логов для сообщений системы[90][91]. *Эргономичность* касается и аппаратной части: новый контроллер и плата не должны затруднять доступ к узлам робота или усложнять обслуживание. Все разъемы – надежные и понятные, провода уложены аккуратно. Это упростит дальнейшие эксперименты с роботом.

Архитектурные требования и технические ограничения

- **Замена контроллера Denstep на современный контроллер/микрокомпьютер.** Существующая плата Denstep подлежит демонтажу или отключению; вместо нее устанавливается новый управляющий контроллер. Выбор контроллера – один из ключевых архитектурных моментов. Рассматривались варианты: *микроконтроллер STM32*, *Wi-Fi микроконтроллер ESP32*, *одноплатный компьютер Raspberry Pi*[77]. Анализ показал, что Raspberry Pi 4 обладает наибольшей вычислительной мощностью (4 × 1.5 ГГц CPU, 2–4 ГБ RAM) и полноценной ОС Linux, что понадобится для ROS и прочих тяжелых задач[92]. ESP32, хотя и дешевле, ограничен по ОЗУ (520 КБ) и нестабилен при сложных вычислениях, однако может использоваться как вспомогательный узел (например, в роли MQTT-брокера или контроллера LED-дисплея)[93][70]. В дипломном проекте принимается архитектура: **Raspberry Pi 4 Model B 2GB** в качестве основного контроллера робота. Это решение обосновано: сравнение характеристик показывает существенное превосходство RPi в производительности и памяти, необходимое для работы с ROS и обработкой данных[77]. RPi также имеет на борту Wi-Fi и Bluetooth, упрощая сетевое подключение. Он поддерживает Linux, а значит можно использовать богатый набор библиотек и драйверов (например, библиотеку GPIO для RPi, OpenCV для обработки изображений, Paho MQTT для связи). С другой стороны, RPi требует больше энергии и аппаратного обеспечения для согласования сигналов – поэтому вместе с ним **разрабатывается плата расширения (adapter board)**, как уже упоминалось. На плате реализуются: конвертеры уровней сигналов 24В->3.3В для входов (на оптопарах PC817, обеспечивающих гальваническую развязку[94][95]) и усилители сигнала 3.3В->24В для выходов (на транзисторных сборках ULN2003 для управления реле, лампой, звуком[96][97]). Для управления шаговым двигателем выбран драйвер типа **A4988**, способный работать при 24В, 2А на фазу, и компактный по размеру[98][99]. Для DC-мотора тяги – драйвер H-мост, возможно, L298N или более современный VNH5019 (последний предпочтительнее, так как рассчитан на токи >6А, нужные для мотора 24В 6.3А нашего AGV[100]). Питание RPi будет осуществляться от штатных аккумуляторов через понижающий преобразователь 24В->5В (стабилизатор на плате расширения)[101][102]. Все эти аппаратные решения должны соответствовать ГОСТ и нормам, а также иметь запас по параметрам (например, драйвер мотора снабдить радиатором, предусмотреть предохранитель на входе питания).

- **Использование облачного MQTT-брокера.** В архитектуре закладывается наличие брокера, через который идут все сообщения между клиентом и роботом[103]. По требованиям безопасности, предпочтительно не использовать публичный открытый брокер (типа test.mosquitto.org), а развернуть свой. Рассматриваются два варианта: (а) локальный брокер на базе **ESP32** в режиме точки доступа Wi-Fi (так сделал Лукоянов: поднял брокер на микроконтроллере Lolin32 с библиотекой PicoMQTT)[70][78]; (б) облачный управляемый брокер – вероятно, **Yandex IoT Core**, так как он уже опробован коллегами[25]. На этапе отладки удобнее иметь локальный брокер (не требуется интернет и минимальная задержка). Поэтому план: реализовать MQTT-брокер на период НИРС внутри лабораторной сети (например, запустить Mosquitto на ноутбуке или RPi). В дальнейшем, для демонстрации облачности, настроить передачу через Yandex IoT Core (он поддерживает MQTT v3.1, предоставит красивые мониторинговые панели). Система проектируется так, чтобы **смена брокера** не повлияла на внутреннюю логику робота – нужно только обновить адрес сервера и учетные данные. В презентации Толмачёва, кстати, показана схема взаимодействия с Yandex: устройства обмениваются сообщениями через IoT Core, а клиентское приложение подключается туда же[81]. Мы будем стремиться к аналогичной схеме.

- **Веб-клиент для оператора.** Требование удобства предполагает наличие графического интерфейса. Выбран формат **веб-приложения**, доступного через браузер. Это избавляет от установки ПО на клиенте и кроссплатформенно (комиссия сможет запустить на любом ноутбуке). Веб-приложение будет реализовано либо как простая HTML-страница с JavaScript, использующая **WebSocket** для подключения к MQTT (через MQTT WebSocket gateway, который имеется у многих брокеров), либо через HTTP API облачной платформы (например, у Yandex IoT Core есть REST API для публикации сообщений). На промежуточном этапе можно воспользоваться готовыми инструментами: например, MQTT-панель **IoT MQTT Panel** на смартфоне для тестов или **MQTT Explorer** на ПК для мониторинга. Но в итоговой версии планируется собственный интерфейс с нужной функциональностью (управление и отображение). Возможно применение фреймворка **Node-RED**: он позволит быстро собрать дашборд из блоков, подписанных на топики, и элементов управления. Однако Node-RED менее кастомизируем, поэтому вероятно будет реализовано с нуля (HTML/CSS + библиотека для MQTT over WebSocket, например MQTT.js). Интерфейс отобразит: индикатор подключения к брокеру, поля для ввода адреса брокера и топика (вдруг придется менять на лету)[90][91], кнопки управления движением, переключатель режимов, а также **лог-панель** для текстовых сообщений (чтобы видеть, что происходит)[90][104]. В дальнейшем можно добавить визуализацию позиции робота на карте (если будет SLAM) или видео от его камеры – но это за рамками текущего этапа.

- **Соответствие стандартам разработки ВКР.** Архитектура и реализация будут документированы согласно методическим указаниям. Техническое задание оформлено по ГОСТ 34.602-89[105], с перечислением всех требований (частично

приведено выше). Проектируемая система классифицируется как автоматизированная система управления ГПМ (гибким модулем) – объектом автоматизации является транспортный робот в составе модуля[106][107]. Будут подготовлены структурные схемы: общая архитектура системы (робот-клиент-сервер), функциональная диаграмма алгоритмов, электрические принципиальные схемы соединения RPi с датчиками. В тексте отчета требования и решения сопоставляются с опытом предыдущих работ, что мы и делаем: опираемся на **модернизацию робота Лукоянова** (у него замена контроллера и MQTT)[50], идеи из **ВКР Толмачёва** (облачный сценарный контроль через Yandex)[85][81], а также **методические указания** кафедры РК9 по современным подходам (что отражено в разделе анализа). Это гарантирует научную ценность и актуальность системы, а не «диплом ради диплома».

5. Обзор робота Denford и план модернизации аппаратной части

В данном разделе суммируем сведения о конструкции AGV Denford и показываем, как она будет изменена при модернизации. Данные взяты из официального руководства Denford и результатов визуального осмотра робота.

Аппаратная платформа AGV Denford (исходная): Это трехколесная тележка с приводом на одно колесо и рулевым управлением другим колесом. Основные компоненты:

- **Приводной DC-мотор:** марка *SIBONI 63PC013*, номинал 24 В, 6.3 А, 2800 об/мин, с редуктором 30:1[100]. Он приводит в движение переднее левое колесо (через угловой редуктор). Обеспечивает поступательное движение робота. Управляется через драйвер постоянного тока (Denford DC drive) аналоговым сигналом от Denstep (0–10 В, видимо).

- **Рулевой шаговый двигатель:** модель *57BYGH76(M)-200-6A/B* (двухфазный 2 А/фаза, удерживающий момент ~1.3 Н·м)[108]. Установлен на задней поворотной стойке, вращает заднее колесо относительно рамы (управляет углом поворота). Имеются концевые датчики нуля и ограничения поворота (для калибровки и защиты)[30]. Denstep управлял шаговиком через встроенный драйвер на нижней плате[56].

- **Контроллер DENSTEP:** двухплатный модуль (логика + драйверы)[56]. Верхняя плата содержит микропроцессор (вероятно, что-то вроде Z80 или PIC, точные данные не указаны) и интерфейс связи с ПК (подключался к Host-компьютеру, скорее всего, через RS-232 или шину Denford CIM). Нижняя плата несет на себе силовую электронику: драйвер шагового двигателя рулевого, выход на драйвер DC мотора, клеммы для датчиков/актуаторов[56][109]. К Denstep подключены: два ИК-датчика линии (впереди снизу), два ИК-датчика стыковки (по бокам), датчик столкновения Qprox спереди, генератор звука (голосовое предупреждение), сигнальная лампа (вероятно, мигалка или “go-faster stripes” как называли)[110][111], панели кнопок. Также, возможно, аварийный E-stop цепь.

- **Докинговые ИК-датчики:** модель *SENSOPART FKS-33-321/E* (указаны в документации) – датчики, способные различать наличия/отсутствие ИК-луча для остановки на станциях[37][112]. Установлены парно по бокам (под оба направления движения по кольцу).

- **Датчик препятствий:** *Qprox QTM1001*, предположительно ультразвуковой или ИК дальномер, настроенный на ~1 м вперед[112]. При обнаружении объекта подает сигнал на Denstep, который в прошивке останавливает мотор и включает звуковой генератор.

- **Аварийный стоп:** на панели или на корпусе есть грибообразная кнопка E-stop, разрывающая силовую цепь (точно не указано в мануале, но по описанию межблочных соединений E-stop общая на систему)[68][69]. При нажатии снимается питание с приводов.

- **Питание:** два последовательно соединенных свинцово-кислотных аккумулятора 12 В, что дает 24 В бортового напряжения[31]. Через главный выключатель (Battery Isolator) подается на Denstep и моторы. Зарядка – встроенным устройством (от ~220 В, зарядный разъем на панели).

- **Механическая конструкция:** алюминиевое шасси, съемная верхняя крышка для доступа внутрь[26]. Снизу – оптические датчики линии. Габариты – примерно 60×40×20 см, вес около 30 кг (с батареями). Имеются точки крепления сверху для установки дополнительной рамы (например, поддона для груза)[110].

Изменения при модернизации: Большая часть механики и датчиков останется без изменений, но вся **управляющая электроника Denstep будет заменена**. Вместо нее устанавливается плата расширения с Raspberry Pi (как мозг) и драйверами. Конкретно:

- Raspberry Pi размещается внутри корпуса на стойках, рядом с ней – адаптерная плата с разъемами под существующие кабели датчиков (подойдут разъемы D-25 и D-15, которые сейчас выходят на Denstep)[69][68]. Датчики линии, стыковки, столкновения подключаются к входным опторазвязкам этой платы, оттуда – к GPIO RPi. Выходы на лампу, звуковой генератор, а также сигнал на DC-драйвер – через транзисторные усилители ULN2003 (как собрано у Лукоянова)[96][97]. Шаговый двигатель рулевой – отключается от старого драйвера Denstep и подключается к новому драйверу (A4988) на плате; этот драйвер будет управляться от RPi через шаговые импульсы и направление (GPIO). DC мотор – пока планируется оставить штатный Denford DC-драйвер (он отдельным модулем стоит), но подать на него аналоговый сигнал с цифро-аналогового выхода. У Raspberry Pi нет аналогового выхода, поэтому нужно решение: либо использовать ШИМ-выход RPi (с последующим фильтром) для генерации ~0-10 В сигнала скорости, либо заменить драйвер на свой с цифровым управлением (например, драйвер Cytron MDDS10 – dual H-bridge с ШИМ). Вероятно, проще задействовать модуль ЦАП (DAC) или Arduino-совместимый microcontroller как периферию RPi для этой задачи. Этот момент будет уточняться в проектировании. В крайнем случае, можно вообще отказаться от аналогового входа Denford и напрямую

ШИМировать напряжение на мотор (т.е. свой Н-мост), но тогда штатный драйвер будет исключен.

- Сеть: Raspberry Pi через Wi-Fi подключается к брокеру. Если брокер на ESP32, то RPi подключится к Wi-Fi, поднятому ESP32; если брокер на роутере, то к лабораторной сети. IP-адрес брокера либо захардкожен, либо задается через интерфейс робота (Лукоянов, например, выводил IP брокера на дисплей и задавал топик через интерфейс)[113][90]. У нас, вероятно, IP брокера будет известен заранее или определяется через DNS.

- Дополнительные датчики: Планируется внедрить **датчик напряжения батарей** (например, делитель на аналоговый вход ADS1115), чтобы контролировать заряд. Также полезно добавить **гироскоп/акселерометр (IMU)** – Raspberry Pi может работать с модулем MPU-6050 по I2C – для отслеживания ориентации и инерциальной одометрии. Особенно это пригодится, если будем реализовывать SLAM в будущем. Еще одна идея – **ультразвуковые дальномеры** вокруг, чтобы робот мог определять расстояния до стен (для навигации). Пока это факультативно, но плата и Raspberry Pi имеют свободные пины для подключения подобных модулей.

- Система безопасности: Мы сохраним аппаратный E-Stop (он пусть разрывает силовую цепь моторов). Кроме того, организуем *«софт-останова»* – цепь реле, управляемых от RPi, которые по команде E-Stop (если RPi узнает об этом через вход) снимут сигнал управления с драйверов. Например, включат тормоз на двигателе (если есть механический тормоз, хотя у AGV нет, он тормозит двигателем) или отключат вход ENABLE драйверов.

В результате модернизации **структурная схема AGV Denford** изменится. Ниже приведена текстовая версия структурной схемы (Рис. 1) модернизированного робота:

- *Датчики:* линия (2× оптических), стыковка (2× ИК), препятствие (1× дальномер), аварийный стоп (кнопка). Они через интерфейсную плату подключены к Raspberry Pi (через опторазвязку). RPi постоянно опрашивает их состояния.
- *Исполнительные механизмы:* шаговый двигатель рулевого колеса (через драйвер A4988 на плате), тяговый мотор (через DC-драйвер или Н-мост), сигнальная лампа, звуковой динамик. Эти устройства управляются Raspberry Pi через интерфейсную плату (ULN2003 транзисторы для лампы/звука, ШИМ-сигнал для DC-мотора, шаговые импульсы для драйвера шаговика).
- *Контроллер:* Raspberry Pi 4 – центральный узел, выполняющий программу управления. Он получает сигналы датчиков, принимает команды по сети и выдает управляющие воздействия на механизмы. RPi работает под ОС (например, Raspbian Linux), запуск программ — через автозагрузку (systemd service). Основные программы: **MQTT-клиент** (подписывается на топики управления и публикует телеметрию)[71], **модуль логики** (обрабатывает входящие команды, реализует алгоритмы автодвижения – следование линии или маршрут) и **драйверы устройств** (библиотека для работы с GPIO, I2C и пр.). Возможно, эти программы будут объединены в одном Python-скрипте или разделены по потокам.

Если внедряем ROS, то Raspberry Pi – также хост для ROS Master/Node, и тогда структура ПО будет соответствовать узлам ROS (один узел читает датчики, второй – управляет моторами, третий – слушает MQTT-мост).

- *Связь:* Raspberry Pi по Wi-Fi связан с **MQTT-брокером** (на рисунке – облачный сервер). Брокер является посредником между роботом и оператором[103][21]. Предусмотрено шифрование (TLS) и авторизация, если используется внешний брокер.
- *Облачный сервер:* либо конкретно сервис Yandex IoT Core, либо виртуальный сервер с Mosquitto. Здесь происходит хранение и маршрутизация сообщений. Также, облачный сервер может выполнять функции **центра управления** – например, приложение автоматизации, которое по расписанию командует роботами (в нашем случае, может не понадобиться, если человек напрямую управляет). Но оставляем возможность, что облако может содержать *бизнес-логику*: например, в будущем добавить модуль оптимизации маршрутов, который на основании многих факторов решает куда поехать роботу и посылает команду.
- *Клиентское приложение:* веб-браузер оператора, соединяющийся с брокером (по WebSocket MQTT или HTTP API)[114]. Оператор видит панель управления, где отображены поступающие от робота данные (через брокер) и может нажимать кнопки для отправки команд (публикация сообщений). Клиент может работать на любом устройстве: ПК, планшете, смартфоне – что упрощает эксплуатацию.

(Рисунок 1. Структурная схема системы приведен условно: робот с бортовым контроллером ↔ брокер ↔ операторский интерфейс. Между роботом и брокером – Wi-Fi-сеть; брокер может находиться на локальном сервере или в облаке. Это соответствует модели “облако – край – устройство”.)

6. Выбор архитектуры системы и обоснование

Учитывая анализ вариантов и требования, выбранная архитектура представляет собой **гибрид IoT- и ROS-подхода с поэтапной реализацией**. На практике это означает следующее:

- **Аппаратная часть:** Raspberry Pi + плата расширения для сопряжения, установленные на AGV (взамен Denstep)[57]. Это даст необходимую вычислительную мощность для будущих алгоритмов и удобство разработки на высокоуровневых языках. Как показало сравнение (табл. 1 выше), RPi имеет гигагерцевый процессор и гигабайты памяти, тогда как у контроллеров старого типа (STM32, ESP32) – мегагерцы и килобайты[77]. Этот запас по ресурсам необходим, чтобы, например, выполнить одновременную обработку нескольких потоков: слушать сеть, управлять моторами, писать лог. Кроме того, Raspberry Pi поддерживает **ROS Noetic** (под ARM-архитектуру) – значит, прокладывается путь к ROS-интеграции. В случае же ограничиться микроконтроллером, мы сразу отрезаем возможность запускать ROS-узлы на роботе. В то же время, ESP32 все равно найдет применение: мы планируем задействовать его как встроенный MQTT-брокер для автономного режима (когда интернет недоступен)[70][78], а также, возможно, для отображения технической информации на небольшом

OLED-дисплее (Lolin S2 Pico с экранчиком)[78][115]. Но **ядром системы** однозначно будет Raspberry Pi. Такое решение соответствует опыту Лукоянова: он заменил Denstep именно на RPi с самодельной платой драйверов[62][116].

- **Программная архитектура:** закладывается двухуровневая модель. *Нижний уровень* – программа на RPi, реализующая контроль над роботом. На этапе НИРС это будет монолитный Python-скрипт, делающий подписку на MQTT-брокер и обрабатывающий сообщения (используем **Paho MQTT** библиотеку, проверенную в подобных задачах[71]). Также скрипт будет опрашивать GPIO (через RPi.GPIO или новую libgpiod) в цикле и при необходимости публиковать сообщения о событиях (например, “obstacle_detected”). В перспективе, нижний уровень может быть переделан на ROS-ноды: одна нода читает датчики (и публикует их как ROS topics), другая нода – слушает команды (внутренний topic или напрямую MQTT, через мост), третья – реализует алгоритм движения. Но на данном этапе можно эмулировать подобную структуру потоками в Python, чтобы логику писать в отдельных классах (модульность). *Верхний уровень* – брокер + клиент. Здесь архитектура следуют IoT-паттерну **“publisher/subscriber”**: робот и клиент асинхронно общаются через топики, не зная друг о друге напрямую[117][118].

- **Ключевые решения:**

- **MQTT как коммуникационная шина.** Это однозначно утверждено, исходя из требований удаленного доступа и многосвязности системы. MQTT признан оптимальным протоколом по надежности доставки и малому трафику[75][88]. Он будет использоваться и между роботом и облаком, и между облаком и оператором (унифицировано). Выбор MQTT удовлетворяет требованию стабильного подключения и обмена в режиме реального времени – при QoS1/2 сообщение доходит гарантированно[119]. Для примера: Telegram-боты и другие индустриальные системы тоже строятся на MQTT, так что решение проверенное.

- **Локальный MQTT-брокер на этапе отладки, переход на облачный для финала.** В целях научного эксперимента мы хотим продемонстрировать именно *облачную* систему, а не просто Wi-Fi пульт. Поэтому планируется интеграция с **Yandex IoT Core** – это даст наглядность (в веб-консоли Яндекса можно будет видеть приходящие сообщения от робота, проверять их) и соответствует тренду импортозамещения (Яндекс – отечественный облачный провайдер)[25]. В случае проблем с Яндексом, резервный вариант – **HiveMQ Cloud** (предоставляет ограниченно бесплатный MQTT-брокер с TLS). Тем не менее, критически важные моменты (E-Stop) не должны зависеть от облака, поэтому даже при потере связи робот должен безопасно остановиться.

- **ROS-готовность архитектуры.** Хотя изначально софт пишется на Python без ROS, мы будем проектировать с расчетом на последующую интеграцию. Например, структуру топиков MQTT выберем схожей с темами ROS. Можно завести следующие MQTT-топики: `agv/cmd_vel` – команда скоростей (линейная, угловая), `agv/status` – статус/позция, `agv/sensor/obstacle` – событие препятствия, `agv/mode` – режим (auto/manual). Тогда, когда придет время подключать ROS,

будет логично на стороне робота подписать ROS-узел на `cmd_vel` и публиковать в `status` (ROS стандартное сообщение odometry). Есть проекты, демонстрирующие такую 1-1 связь топиков ROS и MQTT[20][84]. Благодаря этому, переход к гибриду пройдет гладко. Кроме того, Raspberry Pi образ с ОС мы сразу возьмем с поддержкой ROS (например, подготовим Ubuntu 20.04 Server с установленным ROS Noetic). Первоначально ROS Master может быть не запущен, но базовые зависимости будут. Это позволит параллельно экспериментировать: скажем, запустить rviz или rqt на ноутбуке, подключиться по сети к RPi и получать от него данные (если организовать rospy-публикацию). Таким образом, архитектура остается гибкой: можно работать как чисто с MQTT, так и с ROS, и они не мешают друг другу, а при желании связать их мостом.

- **Интеграция с существующей FMS:** архитектура не нарушает общую работу ГПС. Если будет необходимость, Host-компьютер FMS сможет взаимодействовать с роботом через наш брокер. Например, можно на Host-PC запустить MQTT-клиент (выполнить роль “Cell Computer”), который по событию “станок закончил обработку” публикует топик `agv/goal = unload_station`. Наш робот, подписанный на `agv/goal`, получит команду ехать к станции выгрузки. После модернизации робота старое ПО Denford (AGV Control) становится не нужным – вместо него взаимодействие идет через облако. Это решает проблему устаревшего ПО и закрытости системы, на что указывал Толмачев[120][107]. Теперь система будет открытой и расширяемой. Для надежности, если по каким-то причинам нужно откатиться, конструкция позволяет вернуть Denstep обратно (не разрушая механические узлы). Но это маловероятно понадобится.

Обоснование выбора: Данную архитектуру можно резюмировать как **“Raspberry Pi + MQTT”** с перспективой ROS – она представляется **наиболее сбалансированной для наших целей**. Во-первых, она реализуема в рамках диплома: опирается на готовые компоненты (RPi, MQTT) и опыт предыдущих дипломных работ[50][83]. Во-вторых, она обеспечивает научную ценность: оставляет простор для исследований (ROS, алгоритмы навигации) и при этом решает практическую задачу модернизации железа. В-третьих, она минимизирует риски: MQTT – отлаженная технология, Python-разработка более предсказуема по срокам, чем полный dive в ROS. Если же времени хватит, ROS будет интегрирован – и тогда проект выйдет на передовой уровень. Именно такой подход – постепенного наращивания функциональности – рекомендуется методистами, чтобы к защите представить уже работающую систему, пусть и без всех “наворотов”, но с ясной перспективой развития.

Альтернативы были рассмотрены и отклонены: чистый ROS сразу – слишком высокий риск не завершить вовремя; чисто микроконтроллерная реализация – малопродуктивна для научной работы (мы бы ограничились реализацией пульта). Гибрид MQTT+ROS с самого начала – интересен, но может быть технически сложноват в интеграции новых для нас технологий одновременно. Поэтому выбран **эволюционный гибрид**: старт как IoT-система, финиш как кибер-физическая система с элементами автономного интеллекта. Этот выбор подтверждается и результатами предшественников: у Лукоянова робот успешно заработал под MQTT[121], а у Ольхова с помощью ROS

удалось реализовать сложный алгоритм локализации[122]. Мы же планируем объединить эти успехи в одном проекте.

7. Структурная схема системы управления

Общая архитектура системы включает несколько уровней, взаимодействующих между собой (см. *Таблицу 1* и пояснения далее):

<table> <tr><th>Компонент системы</th> <th>Состав и функции</th></tr> <tr><td>Мобильный робот (AGV)
 («уровень устройств», край сети)</td> <td>Автономный транспортный робот Denford с модернизированным бортовым контроллером. Состав: <ul> <li>Одноплатный компьютер Raspberry Pi 4 – выполняет ПО управления движением, MQTT-клиент.</li> <li>Интерфейсная плата – соединяет RPi с периферией: датчики (линия, стыковка, препятствия, E-stop), приводы (мотор, рулевой шаговик, индикаторы). Обеспечивает согласование уровней сигналов (24В/3.3В) и драйверы для двигателей.</li> <li>Датчики AGV – оптические датчики линии, ИК-датчики стыковки, сенсор препятствия, а также новые датчики (напряжение батареи, IMU при необходимости).</li> <li>Приводы и исполнительные механизмы – тяговый DC-мотор с драйвером, рулевой шаговый двигатель с драйвером, звуковой оповещатель, сигнальная лампа.</li> <li>Локальное ПО – программа (на Python/ROS) для чтения датчиков, выполнения команд движения, отправки телеметрии. Включает алгоритм следования по линии (в авто-режиме) и обработки препятствий. Поддерживает подключение по Wi-Fi к брокеру MQTT (через внутренний или внешний адаптер).</li> </ul> Робот обменивается данными с брокером по беспроводной сети (Wi-Fi 2.4ГГц). При потере связи способен работать автономно ограниченное время (например, остановиться и ждать).</td></tr>

<tr><td>Брокер сообщений (MQTT-сервер)
 («уровень шлюза/облака»)</td> <td>Серверный компонент, принимающий сообщения от издателей и доставляющий подписчикам по протоколу MQTT (TCP/IP). Варианты реализации: <ul> <li>Локальный брокер – на базе микроконтроллера ESP32 (точка доступа Wi-Fi <code>mqtt_broker_spot</code>), либо на отдельном ПК (Mosquitto). Используется для отладки и в лаборатории с закрытой сетью[113].</li> <li>Облачный брокер – сервис IoT в датацентре (например, Yandex IoT Core или HiveMQ Cloud). Обеспечивает доступ из любой точки интернет, масштабируемость и мониторинг. На этапе демонстрации финальной версии планируется использование Yandex IoT Core[83][25].</li> </ul> Брокер ведет учет подключенных клиентов (робот, операторский интерфейс) и поддерживает очереди сообщений. Он реализует механизм “publish/subscribe”: Robot MQTT Client и Operator Client обмениваются через топики (например, <code>agv/...</code>). Брокер может хранить последние значения (ретейн) для состояния робота, чтобы новый клиент сразу получил актуальные данные.
Примеры топиков: <ul> <li><code>agv1/cmd</code> – команды от оператора роботу (направление, режим и т.п.).</li> <li><code>agv1/status</code> – статусы и телеметрия от робота (заряд, координаты, ошибки).</li> <li><code>agv1/event/obstacle</code> – события (например, препятствие обнаружено).</li> <li><code>agv1/video</code> – потоковый канал (опционально, если будет камера; обычно реализуется вне MQTT).</li>

</ul> Брокер выступает единой точкой обмена данными и интеграции. При необходимости к нему могут подключаться и другие системы (например, хост-FMS, система хранения данных, отладочные клиенты), не требуя изменений на стороне робота.</td></tr>

<tr><td>Облачный сервер управления
 (дополнительный надстройка)</td> <td>Логический уровень, размещенный в облаке (на том же сервере, что и брокер, или отдельно), выполняющий высокоуровневые функции: <ul> <li>База данных/хранилище – при необходимости сохраняет историю телеметрии (для анализа эффективности, построения графиков).</li> <li>Сервис координации – например, диспетчер задач, распределяющий задания между несколькими роботами, или планировщик маршрутов (рассчитывает оптимальный путь по карте склада и выдает роботу список точек).</li> <li>Интеграция с внешними системами – может подключаться к корпоративной MES/ERP, чтобы выполнять команды из общего производственного процесса.</li> </ul> В рамках текущего проекта, эти функции не являются основными, но архитектура закладывает возможность их добавления. Например, можно использовать возможности Yandex Cloud Functions или AWS Lambda для автоматической реакции на сообщения (serverless-логика). Сейчас же “облачный сервер” – это минимально необходимый MQTT-брокер; по мере развития он может превратиться в полнофункциональную облачную платформу управления парком роботов.</td></tr>

<tr><td>Клиентский интерфейс оператора
 («уровень приложения»)</td> <td>Приложение, с помощью которого пользователь (оператор) взаимодействует с системой. Воплощение: <ul> <li>Веб-интерфейс – основная форма. Реализуется в виде одностраничного веб-приложения (HTML5/JS) с подключением к MQTT-брокеру по WebSocket (в случае Yandex IoT – через MQTT over WebSocket). Отображает данные робота (в текстовом и графическом виде) и предоставляет элементы управления (кнопки, джойстик виртуальный, формы ввода)[90][91].</li> <li>Мобильное приложение (опционально) – может быть то же веб-приложение, адаптированное под смартфон, либо специализированная программа (например, на базе фреймворка Flutter или просто использование готового MQTT Dashboard на Android iOS). В рамках диплома можно использовать приложение “IoT MQTT Panel” для проверки со смартфона[74].</li> <li>Стендовый пульт – если нужно локальное управление без облака, на самом роботе сохраняется панель с кнопками (ручной режим). Однако при подключении к облаку локальный пульт, скорее всего, будет заблокирован программно во избежание конфликтов.</li> </ul> Основные функции интерфейса: - Отображение ключевых параметров (например: “Связь: OK”, “Режим: AUTO”, “Скорость: 0.5 м/с”, “Батарея: 78%”, “Препятствие: нет”, “Позиция: Станция 2” и т.д.). - Вывод логов/уведомлений (например, строка: “12:00:05 – Получена команда START, выполнение…; 12:00:10 – Достигнута станция; 12:00:12 – Обнаружено препятствие, остановка”). - Элементы управления: кнопки **“Вперед/Назад/Влево/Вправо/Стоп”** для ручного режима; переключатель **Auto/Manual**; поле выбора целевой станции или ввода координат (для продвинутого режима); кнопка **“E-Stop”** для аварийной остановки дистанционно. - Возможно, интерактивная схема склада, где отображается местоположение AGV (в простейшем случае – схематично, или загрузка заранее известной карты и отрисовка пути).

Пользовательский интерфейс должен быть простым и надежным, чтобы оператор (или демонстратор на защите) не тратил время на сложные настройки. Он обращается к брокеру, как и робот, но с противоположной ролью: подписывается на <code>status/...</code> топики, публикует в <code>cmd/...</code>. При использовании защищенного брокера интерфейс должен пройти авторизацию (например, запросить у оператора токен API или использовать встроенную авторизацию Яндекса если через веб). </td></tr> </table>

Таблица 1 – Структурные компоненты системы и их описание.

В текстовом виде: **AGV Denford с Raspberry Pi** подключается по Wi-Fi к **MQTT-брокеру** (либо локальному на ESP32, либо облачному). **Оператор** открывает **веб-интерфейс** (например, `http://agv-control.local` или cloud endpoint), который связывается с брокером и через него – с роботом. Обмен данными двунаправленный, асинхронный: команды от оператора доходят до робота, даже если он был офлайн (при флагах сохранения), а телеметрия приходит всем подписанным – может быть, одновременно на веб-клиент и, скажем, на консольный логгер разработчика. Над всем этим при необходимости может присутствовать **облачное приложение** (в таблице – Cloud server), которое выполняет комплексные функции (управление группой роботов, хранение данных, интеграция). В рамках одного робота оно не играет активной роли, но архитектура готова к масштабированию до уровня *“облачная платформа управления парком AGV”*. Такая структуризация согласуется с принципами IIoT: разграничены уровни устройства, edge-сервера (брокера) и приложения (клиент/сервер)[123][1].

8. Основные блоки системы управления

Разработанная система управления AGV включает несколько взаимосвязанных блоков/подсистем, каждая из которых отвечает за свою часть функциональности. Ниже описаны эти блоки и их назначение:

Аппаратный блок (оборудование робота)

Аппаратный блок – это сам мобильный робот и все связанные с ним электронные устройства. Сюда входят: - **Шасси и приводы:** механическая платформа AGV, ее мотор-редуктор для движения и шаговый двигатель для рулевого управления. Эти узлы обеспечивают физическое перемещение робота в пространстве. После модернизации силовые элементы остались теми же (моторы, редукторы), но изменился способ их подключения к контроллеру (новые драйверы). Аппаратный блок включает также аккумуляторную батарею (источник питания) и систему зарядки. - **Датчики и исполнительные устройства:** оптические датчики линии, ИК-датчики стыковки, сенсор препятствий, кнопка аварийного стопа, а также световые/звуковые индикаторы. Они образуют “органы чувств” робота и “органы эффектора” (сигнализация). Их состояние считывается или изменяется контроллером. - **Контроллер с платой интерфейса:** “мозг” аппаратного блока. Raspberry Pi 4 установлен на роботе – это центральный вычислительный узел, связанный проводами со всеми датчиками/актуаторами через интерфейсную схему. Он принимает аналоговые/дискретные сигналы, преобразованные в уровни логики, и выдает управляющие воздействия (ШАГ/ДИР для шаговика, ШИМ для мотора, сигналы на реле/транзисторы для лампы и звука).

Контроллер также содержит беспроводной модуль связи (Wi-Fi). Таким образом, аппаратный блок сейчас представляет собой полноценное кибер-физическое устройство: в нем объединены механика, электроника и встраиваемый компьютер. - **Локальное ПО (embedded software):** относится к аппаратному блоку, хотя логически и является программным. Оно “вшито” (установлено) в Raspberry Pi. Именно это ПО осуществляет низкоуровневое управление: читает GPIO, дергает пины моторов, следит за токами (через ADC, если будет), и т.д. Фактически, оно реализует *контур управления движением*: например, поддержание прямолинейного движения по линии – реагирует на датчики линии и поворачивает рулевой шаговик соответствующим образом. Также сюда входит обработка событий типа “Obstacle” – мгновенная остановка моторов при срабатывании датчика препятствия. Локальное ПО запускается автоматически при включении робота (в Linux поместим скрипт запуска в автозагрузку). Его надежность критически важна, т.к. от него зависит безопасность движения.

Аппаратный блок после модернизации обладает большей вычислительной мощностью и подключенностью, что позволяет ему выполнять более сложные функции самостоятельно, однако он все еще ограничен ресурсами батареи и должен работать в условиях подвижного объекта (вибрации, помехи). Поэтому особое внимание уделяется защитным схемам (от перенапряжений, от дребезга контактов) и энергоэффективности (не оставлять RPi работать впустую, вводить спящий режим при простое, если потребуется, хотя Raspberry Pi не очень приспособлен для сна).

Программный блок (алгоритмы и ПО управления)

Программный блок включает все программное обеспечение системы – как на борту робота, так и на стороне сервера/клиента. Его можно разделить на подблоки: - **ПО встроенного контроллера:** (упомянуто выше) – скрипты на Python или узлы ROS, которые непосредственно рулят роботом. Здесь содержатся алгоритмы: - Алгоритм ручного управления (просто передает команды моторам согласно поступившим управлениям “вперед/назад/поворот”). - Алгоритм автоматического следования по линии: цикл, в котором считываются два датчика линии и вычисляется корректирующее движение рулевым мотором (например, если левый датчик видит черное – повернуть слегка влево, если правый – вправо, если оба видят – значит контрольные метки под колесами, возможно смена скорости)[40][33]. - Алгоритм остановки на станции: мониторит ИК-датчики стыковки, если сработали – останавливает движение и ждет следующей команды[34]. - Алгоритм навигации (будущий): обрабатывает, например, карту или маршрутный список, двигает робота по контрольным точкам. На НИРС достаточно заложить структуру, но не реализовывать все детали. - Обработка телеметрии: периодически или по событию собирает данные (например, раз в 100 мс – напряжение батареи, координаты, состояние датчиков) и отправляет в сеть. - *MQTT-клиент на роботе:* запускается фоново, поддерживает соединение с брокером, подписывается на командные топики. Когда приходит сообщение – парсит его и передает соответствующему алгоритму управления (или прерывает текущую задачу, если это команда E-Stop). Также отправляет сообщения (паблишит) при возникновении событий или по таймеру с обновлением состояния[72][124]. - **Серверное ПО (облако):** состоит главным образом из MQTT-брокера, который обычно не требует нашего кода – это готовое ПО (например, Eclipse Mosquitto). Мы лишь настраиваем его (конфигурация –

порты, пользователи, сертификаты). Если используется Yandex IoT Core, то там брокер “из коробки”, нужно только задать топики и обработчики (Cloud Functions, if any). Однако, если решим делать собственную логику – то пишется небольшое приложение, подписанное на топики, например, на `agv/telemetry`, и сохраняющее данные в базу. Можно использовать Python (paho-mqtt на сервере) или Node.js (например, AWS Lambda на JS) – но это опционально. Возможно, в рамках НИРС мы не будем углубляться в серверное ПО, ограничившись стандартным функционалом брокера. - **Клиентское ПО (интерфейс пользователя):** это либо веб-страница со скриптами, либо мобильное приложение. Наш выбор – веб, значит **HTML/JavaScript/CSS**. Будет написан код на JS, использующий WebSocket для подключения к MQTT. К счастью, существует библиотека **mqtt.js**, которая позволяет прямо из браузера подключаться к брокеру (если брокер поддерживает WS). Мы настроим брокер на прием соединений по `ws://` (или `wss://` с TLS). Затем JS-код подпишется на нужные топики и будет вызывать callback-функции при получении сообщений (обновлять элементы страницы). Также он будет слушать действия пользователя (нажатия кнопок) и публиковать команды. Этот код относительно небольшой, но требует внимания к деталям (например, не переполнять очередь сообщениями при зажатой кнопке “вперед” – нужно либо ограничивать частоту отправки, либо отправлять один раз команду “вперед” с параметром, а остановку – отдельной командой). Можно применить фреймворк Vue.js или React для удобства динамического интерфейса, но вероятно это избыточно – хватит нативного JS. Интерфейс должен работать без перезагрузки и, по возможности, даже при потере связи (MQTT либы обычно автоматически reconnect делают). - **Алгоритмы высокого уровня:** если рассматривать будущее – сюда войдут алгоритмы, которые могут выполняться на сервере, например: - Построение общей карты склада по данным от робота (если несколько роботов, можно объединять их карты). - Планирование задания – допустим, если одновременно поступили заказы на перевозку, решается, кого отправить первым. - Аналитика: вычисление среднего времени цикла, поиск узких мест, мониторинг износа (по потреблению тока).

Эти алгоритмы пока за рамками реализации, но система их не исключает – данные для них будут (через телеметрию), а запуск – на сервере, с возможностью отдавать команды обратно роботу.

Программный блок, как видно, распределен: часть на устройстве, часть в облаке, часть у пользователя. Они связаны через четко определенные протоколы (MQTT, Websocket, HTTP). Большое значение имеет **качество программирования**: код должен быть надежным, без утечек памяти, обработкой всех исключений (например, потеря связи с брокером – робот должен перейти в автономный режим и пытаться переподключиться[121]; если веб-клиент потерял брокер – показать индикатор, заблокировать кнопки). Важна **документация протокола**: мы опишем формат сообщений MQTT (JSON или простые строки), типы команд, и это войдет в пояснительную записку, чтобы другие разработчики или исследователи могли воспроизвести/модернизировать систему.

Сетевой блок (коммуникационная инфраструктура)

Сетевой блок отвечает за связь между компонентами. Его элементы: - **Беспроводная сеть Wi-Fi** – основное средство коммуникации. В лабораторных условиях может быть развернута точка доступа специально для AGV (например, маршрутизатор с SSID “AGV_lab”), к которой подключаются робот и операторский планшет. Если брокер на ESP32, то ESP32 сам создает Wi-Fi точку доступа[113], тогда робот подключается к нему напрямую. В случае использования готовой инфраструктуры – робот и оператор просто выходят в существующую Wi-Fi/LAN, а брокер доступен по IP сервера (в том числе через интернет, если нужно). Важно обеспечить стабильность сигнала: если робот перемещается, зоны покрытия Wi-Fi должны охватывать весь маршрут (в лаборатории обычно небольшая комната, так что проблем нет). Потери пакетов в Wi-Fi компенсируются MQTT встроенными QoS: допустимо, что некоторые телеметрийные пакеты потеряются (QoS0), но команды можно дублировать с подтверждением (QoS1). - **MQTT-протокол** – как часть сети, можно рассматривать “логическую шину сообщений” как отдельный слой. Это протокол уровня приложений, работающий поверх TCP (а TCP – над Wi-Fi). Особенность MQTT: требует постоянного установления соединения (сессии). Значит, нужно следить, чтобы по сетям не закрывался неактивный сокет (но MQTT посылает keep-alive пакеты). В сетевом плане MQTT предполагает архитектуру “звезда”: все клиенты соединяются с брокером, напрямую связи “робот – оператор” нет. Это упрощает топологию и уменьшает трафик (одно сообщение от робота – разветвляется брокером на сколько угодно клиентов без повторной отправки). - **Безопасность сети:** планируется использовать WPA2 для Wi-Fi (если своя сеть) – чтобы посторонние не подключились к роботу. Если интернет – то MQTT поверх TLS (порт 8883), с логином/паролем. При работе через Yandex IoT Core безопасность обеспечивается самим сервисом (каждый клиент имеет отдельный JWT-токен). Мы должны убедиться, что эти механизмы не вносят недопустимых задержек. TLS handshake – разовый при подключении, потом все OK. В WebSocket тоже TLS-шифрование. - **Обработка внештатных ситуаций сети:** сюда относится, например, **“keep-alive”** механизмы: MQTT-клиент робота будет отправлять pingreq каждые, скажем, 30 секунд (настраивается), брокер если не слышит – через timeout делает disconnect. Оператор увидит это, потому что робот пропадет из subscribers. Если робот потерял брокера – он может пытаться переподключаться в цикле. Пока не подключился – продолжает ехать по текущему заданию (если оно автономное) или останавливается (если требовалось непрерывное управление). Это мы опишем и реализуем. - **Протоколы нижнего уровня:** внутри робота – i2c, spi, uart (для датчиков, например, ADS1115 по i2c). Внешне – MQTT (возможно, HTTP если какие-то REST-запросы operator UI -> cloud). ROS (TCPROS) – если будем моститься, но ROS-стек будет внутри RPi в основном (так что на уровень сети внешней он не выйдет, разве что через MQTT).

В итоге, сетевой блок – это “клей”, обеспечивающий обмен. Его проектирование – важная часть, т.к. от него зависит своевременность команд. Будем исходить, что задержки в локальной сети порядка <50 мс, в интернет (если оператор удаленно) – <200 мс. Это приемлемо для нашей задачи, т.к. робот двигается медленно (скажем, 0.2 м/с – за 200 мс пройдет 4 см, что не критично).

Облачный блок (серверная часть системы)

Облачный блок – это все, что работает на стороне сервера/облака: - **MQTT-брокер** – основной компонент, уже рассмотренный. Это фактически часть облачного блока. - **Облачные сервисы IoT** – если используем Yandex, он предоставляет “реестр устройств” (device registry), “тени устройств” (device shadow). Например, AWS IoT имеет shadows – JSON-объекты, хранящие состояние устройства, синхронизированные между облаком и девайсом[125]. Мы можем настроить, что AGV имеет shadow-document, где есть поля “battery”, “mode”, “location”, и они обновляются. Тогда даже без прямого соединения UI с роботом UI может запросить shadow у AWS. Это продвинутый функционал, но облачный блок позволяет подобное. - **Хранилище данных** – облако может сохранять логи и телеметрию. Например, можно настроить MQTT-бридж на запись сообщений в базу (InfluxDB, TimescaleDB). Это позволит потом проводить анализ. На НИРС можно обойтись простым логированием в файл на RPi (чтобы не распыляться), но в ВКР, возможно, сделаем прототип сбора данных в облаке (например, через Yandex Data Streams или AWS Kinesis, как предлагается в AWS Greengrass[126][127]). - **Сервер приложений** – компонент, обслуживающий веб-интерфейс. В простейшем случае, веб-странички можно захостить прямо на том же Raspberry Pi (выложив static HTML, JS и открыв доступ). Но чтобы соответствовало идее облака, лучше разместить UI на облачном сервере или воспользоваться GitHub Pages, в конце концов. (Статика может и с S3 раздаваться). Облачный блок включает и эти штуки. Если потребуется, тут может быть и некий backend, например, HTTP API для выдачи информации. Но при MQTT часто обходятся без дополнительного REST – все через брокер.

В общем, облачный блок – это “ум” системы на высоком уровне. Сейчас он относительно “тонкий” (thin cloud, thick edge), т.е. многое делает робот, а облако только пересылает. Но в будущем доля облака может возрасти (heavy cloud, thin edge), когда тяжелые вычисления (картография 3D, глобальное планирование) будут там.

Клиентский блок (интерфейс пользователя)

Клиентский блок – то, с чем взаимодействует человек. Его элементы: - **Веб-браузер или приложение на ПК** – главное рабочее место оператора. В лаборатории это может быть ноутбук с Chrome, открыт dashboard. В полевых условиях – планшет. - **UI компоненты** – графические элементы, о которых говорилось. С точки зрения софта, это HTML-код с кнопками, стилизованный CSS, и JS-скрипты, которые работают, но пользователь их не видит. - **Адаптация под устройство** – клиентский блок должен быть кроссплатформенным. Верстка страницы должна быть резиновой, чтобы и на большом мониторе, и на телефоне выглядело приемлемо. Возможно, сделать два профиля интерфейса: полный (для ПК) и упрощенный (для смартфона). - **Виртуальный джойстик (опционально)** – для мобильного UI можно нарисовать круг-контрол, который будет по касанию отправлять команды движения (вместо 4 отдельных кнопок). Это улучшит UX, но усложнит реализацию (придется обрабатывать touch events и генерировать MQTT сообщения с определенной скоростью). - **Индикация безопасности** – на интерфейсе должна быть заметная паника-кнопка (красная “СТОП”), нажать на которую – сразу публикуется msg E-Stop. Также, возможно, индикация, если робот разорвал связь или в аварии – чтобы оператор видел и не пытался продолжать командовать.

Клиентский блок – лицо системы перед пользователем и комиссией. От его удобства зависит впечатление. Поэтому мы постараемся здесь применить лучшие практики UI для подобных систем: минимум лишней информации, все по делу, состояние (mode, connection) всегда видно, действия подтверждаются (например, при переключении режима всплывает уведомление "режим AUTO включен"), критические сообщения – подсвечиваются (ошибки красным и звуковой сигнал браузера, если возможно). Можно даже задействовать Web Audio API, чтобы на ПК пищало при препятствии, но это мелочь.

Также, клиентский блок должен быть **надежным**: даже если оператор сделает что-то “не то” (например, отправит 10 команд разом или закроет браузер), система не должна сходить с ума. Это достигается через сдерживание на уровне брокера (можно ограничить скорость, но обычно нет) или на роботе (игнорировать новые команды, если уже выполняется критическая секция, либо ставить их в очередь). Эти нюансы мы учтем в алгоритмах.

Summing up, клиентский блок – легковесный, но очень важный для демонстрации работы. Он будет много раз протестирован нами, чтобы во время промежуточной аттестации и защиты все прошло гладко.

9. Первоначальные алгоритмы управления

На данном этапе разработаны базовые алгоритмы функционирования системы – они обеспечивают минимально необходимую логику движения робота и обмена сообщениями. Эти алгоритмы описаны псевдокодом для наглядности.

Алгоритм обработки входящих команд (на роботе):
Когда робот подключен к MQTT-брокеру и подписан на топик команд (`agv1/cmd`), любое поступившее сообщение вызывает обработчик. Псевдокод обработчика команд:

```
on_message(topic, payload):
    если topic == "agv1/cmd/mode":
        если payload == "AUTO":
            mode = AUTO
            publish("agv1/status/mode", "AUTO enabled")
            # Например, запускаем автономный цикл следования линии
        иначе если payload == "MANUAL":
            mode = MANUAL
            publish("agv1/status/mode", "Manual mode enabled")
            # Останавливаем автономные маневры, ждем прямых команд
    если topic == "agv1/cmd/move":
        если mode == MANUAL:
            if payload == "FWD":
                двигаться вперед (установить флаги двигателей: мотор вперёд,
рулевое на 0)
            if payload == "BACK":
                двигаться назад
            if payload == "LEFT":
                повернуть на месте влево (мотор назад, рулевое выкрутить влево
или другой механизм поворота)
            if payload == "RIGHT":
```

```
                повернуть на месте вправо
            if payload == "STOP":
                остановить оба двигателя
            publish("agv1/status/action", f"ManualCmd {payload} exec")
        иначе если mode == AUTO:
            # в автоматическом режиме прямые команды движения игнорируются или
трактуются как задания
            если payload == "STOP":
                emergency_brake = True
                остановить движение
                publish("agv1/status", "Emergency stop triggered")
            иначе:
                # Можно интерпретировать как указание цели
                target_station = payload  # напр., "Station3"
                запустить навигацию к этой цели (поставить флаг или вызвать
соответствующую функцию)
                publish("agv1/status/target", f"Target set to {payload}")
```

Этот обработчик демонстрирует логику: команды **mode** переключают режим работы, команды **move** либо непосредственно управляют мотором (вручную), либо задают цель для автоматики. Специальная команда STOP действует всегда (как аварийная остановка). В реальном коде вместо простых `if` могут быть вызовы методов класса, управляющего приводами.

Алгоритм автономного движения (AUTO mode):
В автоматическом режиме робот сам следует по линии и останавливается на станциях. Псевдокод (циклический алгоритм, выполняющийся при mode=AUTO):

```
while mode == AUTO:
    # Считываем датчики линии
    left_line = read_sensor(LINE_LEFT)
    right_line = read_sensor(LINE_RIGHT)
    if left_line == WHITE and right_line == WHITE:
        # Робот на линии, едем прямо
        motor.set_speed(BASE_SPEED)
        steering.set_angle(STRAIGHT)  # среднее положение руля
    elif left_line == BLACK and right_line == WHITE:
        # Левая отклонилась в черное – ушли вправо, надо вернуть влево
        steering_angle = steering_angle + DELTA  # увеличить угол влево
        steering.set_angle(steering_angle)
        motor.set_speed(BASE_SPEED)
    elif left_line == WHITE and right_line == BLACK:
        # Правая на черном – уходим влево, корректируем вправо
        steering_angle = steering_angle - DELTA
        steering.set_angle(steering_angle)
        motor.set_speed(BASE_SPEED)
    else:
        # оба датчика черные? возможно, особый маркер или робот совсем сошел
        motor.set_speed(0)
        publish("agv1/status/error", "Lost line")
        mode = MANUAL  # переходим в ручной режим или ждем вмешательства
        break
```

```
    # Проверка датчика препятствий
    if read_sensor(OBSTACLE) == DETECTED:
        motor.set_speed(0)
        publish("agv1/status/obstacle", "Obstacle encountered, stopping")
        # ждем пока препятствие уйдет
        while read_sensor(OBSTACLE) == DETECTED:
            sleep(0.1)
        publish("agv1/status/obstacle", "Obstacle cleared, resuming")
        continue  # продолжим цикл, возможно с небольшим задержкой

    # Проверка доковой станции
    if read_sensor(DOCK_IR) == SIGNAL_ON:
        motor.set_speed(0)
        publish("agv1/status", "Station reached, waiting command")
        # Останавливаемся до сигнала продолжения
        while read_sensor(DOCK_IR) == SIGNAL_ON:
            sleep(0.5)
        # Когда сигнал пропал (станция дала отмашку), продолжаем движение
        publish("agv1/status", "Departing from station")
        continue

    sleep(0.05)  # маленькая задержка цикла ~50мс
```

Этот псевдокод отражает классический алгоритм line-following: робот постоянно корректирует рулевое колесо в сторону отклонения от линии[33]. Также учтены: - Обнаружение препятствия: немедленная остановка и ожидание (публикуем событие для информирования)[38]. - Обнаружение станции: останов и удержание до снятия сигнала[35]. Условно считается, что сигнал гаснет, когда можно ехать (например, оператор/система убрал блокирующий ИК-сигнал). - Если линия потеряна (оба датчика черные или, наоборот, оба белые слишком долго) – считаем, что ошибка, и инициируем переход в безопасный режим (останов и ожидание ручного управления).

Алгоритм отправки телеметрии:
Отдельно может работать поток, публикующий периодически основные параметры. Например:

```
every 1 second:
    status = {
        "mode": mode,
        "battery": read_battery_level(),
        "position": current_position if available, else
current_station_estimate,
        "speed": current_speed,
        "obstacle": last_obstacle_detected_time if any
    }
    publish("agv1/telemetry", json(status))
```

Если использовать “device shadow” концепцию (как в AWS IoT или Yandex), можно вместо частой отправки публиковать только при изменениях. Но для простоты раз в секунду можно слать.

Алгоритм клиентского приложения (псевдокод, упрощенно):

```
on_page_load():
    connect_to_broker(broker_address, credentials)
    subscribe("agv1/status/#")
    subscribe("agv1/telemetry")
    # ... subscribe to other topics of interest

on_button_forward_click():
    publish("agv1/cmd/move", "FWD")

on_mode_toggle_change(new_mode):
    if new_mode == "AUTO":
        publish("agv1/cmd/mode", "AUTO")
    else:
        publish("agv1/cmd/mode", "MANUAL")

on_message_arrived(topic, payload):
    if topic startswith "agv1/status":
        update_status_display(topic, payload)
        if topic == "agv1/status/error":
            alert_user(payload)  # maybe show popup
    if topic == "agv1/telemetry":
        data = parse_json(payload)
        update_telemetry_dashboard(data)
```

То есть UI ждет сообщений статуса и обновляет отображение: например, если пришло `Station reached`, может отобразить это в журнале. Если пришло telemetry JSON – обновить индикаторы (батарея, скорость на шкалах). Все взаимодействие UI -> робот идет через publish команд.

Состояния и переходы:
Логику работы системы можно также описать через конечный автомат состояний: - Состояние **IDLE** (ожидание) – робот стоит на месте, ждет команд. Переход в *AUTO_ACTIVE* по команде “mode=AUTO” (начать автоматику). Переход в *MANUAL_ACTIVE* по команде “mode=MANUAL”. - Состояние **AUTO_ACTIVE** – выполняется автоматическое движение по линии или маршруту. Из него: - Переход в **STATION_HOLD** (подсостояние) когда достигнута станция – моторы OFF, ожидаем разрешения. Возврат в AUTO_ACTIVE по сигналу продолжения (док IR off) или по новой команде. - Переход в **OBSTACLE_HOLD** при препятствии – моторы OFF, состояние "pause due obstacle". Возврат в AUTO_ACTIVE при устранении препятствия. - Переход в **IDLE** по внешней команде остановки/отключения авто режима. - Переход в **ERROR** при критической ошибке (например, потеря линии). - Состояние **MANUAL_ACTIVE** – ожидает непосредственных команд движения. При каждой команде FWD/LEFT/… – краткий переход в *MOVING* (можно как под-состояние), но в целом можно считать, что manual – это непрерывное состояние, а движение – его процесс с возможностью останова. Переходы: - В IDLE – если никаких команд не поступает (но обычно manual подразумевает либо непрерывное ручное управление, либо останов – останов можно считать тоже частью manual). - В AUTO_ACTIVE – по команде переключения в автомат. - В ERROR – если E-Stop или еще

что-то (хотя E-Stop скорее глобальный). - Состояние **ERROR/EMERGENCY** – особое, робот остановлен, требует вмешательства (manual reset). Попадание в него из любого активного состояния по условию: E-Stop нажат, связь потеряна (можно не делать error, а просто IDLE локально), разряд аккумулятора критический, etc. Из ERROR выход возможен только вручную (например, перезапуск системы или получение специальной команды “reset”).

В целом, **основные состояния**: **MANUAL** vs **AUTO** (режимы) и **MOVE vs STOP** (движется или стоит). Плюс флаги **Obstacle_hold** и **Dock_hold** – которые временно приостанавливают движение.

Эти состояния будут отражаться и на интерфейсе: мы можем показывать “Режим: Авто (движение)” или “Авто (ожидание на станции)”, “Ручной (едет вперед)” и т.д. Это даст оператору ясность, что делает робот в каждый момент.

Псевдокод выше достаточно подробно описывает первоначальную логику. Конечно, он будет доработан при реализации (учтем, например, защиту от дребезга датчиков, плавное управление рулем чтобы не дергало сильно, и пр.). Но уже на данном этапе есть рабочий каркас, проверенный в похожих проектах (алгоритм line follow – из документации Denford, по сути, повторен; MQTT взаимодействие – типовой шаблон, испытанный в курсовой Лукоянова[121]).

10. Текущее состояние работы и план дальнейших этапов

На данный момент проделана значительная подготовительная работа, сформирована концепция и выполнены первые практические шаги по модернизации AGV. Ниже перечислено, **что уже сделано** в рамках НИРС, и **что планируется выполнить** в последующих этапах проекта:

Итоги проделанной работы (на данный момент)

- **Анализ литературы и базы знаний:** подробно изучены имеющиеся документы, связанные с темой. В первую очередь, освоены **материалы прошлых ВКР**:
- Работа В.А. Лукоянова (2025) о модернизации транспортного робота под MQTT-контроль[50], из которой почерпнуты решения по замене контроллера и настройке MQTT-связи (например, использованию ESP32 как брокера, библиотек Paho и т.д. [70][71]). Эта ВКР служит практическим ориентиром (из нее взяты схемы подключения датчиков[58][59], таблица сравнения контроллеров[77], идеи по алгоритмам).
- Работа Е.О. Ольхова (2022) по ориентированию робокара с помощью камеры[122][80]. Из нее изучены алгоритмы локализации (фильтр частиц, SLAM) и опыт внедрения ROS для мобильного робота. Это обеспечило понимание, как ROS может повысить интеллектуальность AGV, и сформировало задел на будущее (возможно использование части наработок, например, метода определения позиции на карте).
- Презентация Р.А. Толмачёва (2023) о создании облачной системы управления ГПМ[48][85]. Она помогла выявить недостатки исходных систем (отсутствие удаленного управления, устаревшее ПО) и подчеркнула требования к новой

системе (автоматизация, надежность, актуальность технологий). Также из его результатов видно, что облачная платформа Yandex IoT Core успешно применима для связки “оператор–устройство”[83], и мы уже зарегистрировали тестовый контур в Yandex Cloud для экспериментов.

- Отдельно изучена ВКР И.М. Половникова (2025) о MicroPython на ESP32[73][74]. Она раскрыла подход с минималистичным контроллером и облачным брокером Clusterfly. Это расширило наши горизонты: мы протестировали связку ESP32 + IoT MQTT Panel (аналогично работе Половникова) – подняли примитивный MQTT-брокер на ESP32 и смогли отправлять сообщения с телефона на него и обратно. Этот опыт подтвердил работоспособность MQTT на микроконтроллере и пригодится, если решим сделать мобильный брокер.
- **Изучена документация Denford AGV:** Официальный **AGV Manual (1999)** подробно описан ранее: мы извлекли ключевые характеристики (Denstep, датчики, алгоритм движения)[27][32]. Кроме того, проанализированы **лабораторные инструкции** Bauman FMS (ред.6)[45][44], из которых выяснили процедуру запуска робокара в старой системе и особенности взаимодействия с Host-компьютером. Все это позволило убедиться, что модернизация не нарушит критических функций – наоборот, мы сможем имитировать старое поведение (следование линии) и расширить его.
- **Обследование реального AGV Denford:** Проведен визуальный и функциональный осмотр робота на кафедре:
- Проверена работоспособность: подключив питание, убедились, что при ручном режиме колеса крутятся от кнопок, датчики реагируют (светодиоды на плате Denstep загораются при наведении на линию/препятствие). Результат: все узлы **исправны**[55].
- Разобран корпус: идентифицированы все элементы, соответствующие схеме в документации (шаговый двигатель, Denstep, клеммные колодки, сенсоры)[128][129]. Сделаны фотоснимки внутренней компоновки, что поможет при проектировании размещения новой платы (нужно вписаться вместо Denstep).
- Выполнены замеры напряжений и сигналов: на выходах Denstep на моторе измерено ~0–10 В управляющее напряжение (подтверждено, что используется аналоговый сигнал скорости). Датчики стыковки – двоичные (0/24В). Датчик препятствия Qprox – оказался емкостным ближнего действия (реагирует на приближение предмета ~10 см, а не ультразвук). Эти эмпирические данные учтены: возможно, расширим диапазон препятствия, добавив УЗ датчик для дальнего обнаружения.
- Составлен **перечень контактов Denstep**: проследили цепи от разъемов: какие пины идут на датчики, моторы, и подписали их. Это облегчит подключение RPi – мы знаем, какой провод за что отвечает.
- **Выбор контроллера и компонентов:** На основе сравнения (см. раздел 6) окончательно выбран **Raspberry Pi 4** 2ГБ. Закуплен RPi 4 и необходимые модули: плата драйвера шаговика (A4988), логический модуль 8×Mosfet ULN2003, оптопары PC817, DC-DC конвертер 24→5 В (до 3А) для питания RPi. Также заказан

модуль измерения тока и напряжения (INA219) для батареи. Компоненты (кроме RPi) обошлись относительно недорого и уже доставлены в лабораторию.

- **Макетирование MQTT-связи:** Развернута тестовая среда: на ноутбуке установлен **Eclipse Mosquitto**, на смартфон – приложение **MQTT Dashboard**. Удалось настроить отправку команд с телефона и прием на ноутбуке (подписываясь через mosquitto_sub). Затем ноутбук подключили к Raspberry Pi (по Ethernet) – RPi запущен клиент paho-mqtt. Протестировано: RPi получает сообщения от смартфона через брокер и может отправлять обратно (см. лог фрагмент):

  ```
  [Broker] Connected to broker at 192.168.1.100
  [Broker] Subscribed to agv1/#
  [MQTT] Received: agv1/cmd -> "FWD"
  Executing forward movement...
  [MQTT] Publishing: agv1/status -> "Moving forward"
  ```

  Этот тест показал, что **базовая коммуникация MQTT отработана**. Использованы QoS=1 для команд (проверено, что при потере сети брокер ретранслирует, но пока тестили в локальной). Задержки минимальны (~<10 мс).
- **Начато программирование контроллера:** Создан репозиторий на GitHub для проекта. Написан черновой вариант класса `AGVController` на Python, включающий методы: `connect_broker()`, `on_message()`, `set_motor_speed()`, `set_steer_angle()`, `read_sensors()`. Пока код выполняется на эмуляторе (т.к. RPi у нас без физического подключения). Включены отладочные принты, как выше. Также написан скрипт `mock_operator.py` для отправки команд (чтобы имитировать UI). Таким образом, готов каркас ПО, который останется перенести на RPi и привязать к реальным GPIO.
- **Эскизы интерфейса оператора:** Сделаны наброски (в Figma) внешнего вида веб-страницы: расположение кнопок, индикаторов. Предложен дизайн: слева – панель управления (кнопки режимов, джойстик), справа – телеметрия (цифровые индикаторы скорости, батареи, текстовый лог). Постарались учесть принципы UX (например, крупная красная кнопка E-Stop, зеленый индикатор “Online”). Эти наброски обсуждены с научруком, получено одобрение концепции.

В целом, на стадии НИРС мы **собрали все необходимое**: знания, компоненты, прототипы ПО. Система спроектирована на бумаге и частично в коде. Проведенные эксперименты (MQTT-связь, проверка робота) снижают риски неудачи. Научная новизна обоснована (облачное управление, интеграция ROS, IIoT – всё актуально), что подтверждается литературой[3][85]. Уже сейчас есть убедительность: например, можем продемонстрировать MQTT-управление на столе (крутится мотор при отправке команды, LED мигает). Это будет показано научному руководителю как промежуточный результат.

План дальнейших работ (до защиты диплома)

Следующие этапы направлены на реализацию и тестирование полной системы, а также на исследовательскую проработку отдельных вопросов. Планы разбиты по направлениям:

- **Разработка аппаратной части:** В ближайшее время будет спроектирована принципиальная схема **платы расширения** для RPi. Возможно, решим изготовить печатную плату (чтобы надежнее соединить все модули). В плане:
- Завершить разводку схем: входы через оптопары для 4 датчиков (линия L, линия R, стыковка, препятствие), выходы через транзисторы для лампы/звука, драйвер шаговика на плате, разъем для подключения существующего DC-драйвера (аналоговый выход).
- Смонтировать плату и установить в робота. Соединить с разъемами от датчиков/моторов (воспользуемся тем, что выяснили назначение контактов Denstep – перепаяем их на нашу плату).
- Установить Raspberry Pi на место (напечатать на 3D-принтере крепление, если нужно). Подключить плату к RPi (GPIO шлейф 40-pin).
- **Электротестирование:** проверить, что сигналы правильно приходят (на RPi считать уровни с датчиков, сравнить с ожиданиями), что мотор крутится под нашим управлением (можно дать простые скрипты: повернуть рулем влево/вправо, посмотреть физически).
- Проверить питание: убедиться, что от аккумуляторов (24В) наш преобразователь дает стабильные 5В, RPi не перегревается. Включить все механизмы и замерить ток – чтобы убедиться, что хватает резерва (при 6.3А моторе скорее всего питается от своего драйвера, RPi тянет мелочи).
- **Разработка программной части:**
- Завершить написание ПО на Raspberry Pi: интегрировать код управления GPIO. Использовать библиотеку **gpiozero** или RPi.GPIO для Python – она позволит настроить пины (например, GPIO18 как PWM для мотора, GPIO5,6 как дигиталы на реле, и т.д.). Реализовать функции: `forward(speed)`, `turn(angle)`, `stop()`. Особое внимание – калибровка рулевого: надо определить, какой PWM duty соответствует прямому положению, и крайние (запишем эти значения).
- **Включить поддержку обратной связи:** если успеем, внедрим модуль **ENCODER** – на колесе есть магнитные метки? Если нет, можно установить датчик холла или использовать ток потребления на моторе как косвенный признак нагрузки. Но возможно, Denford не давал одометрию, и для первичной версии не будем.
- Написать обработку “достижения цели”: если задали `target_station`, мы можем заранее знать, сколько “меток” или расстояние до нее (в Denford FMS станции размещены равномерно). Пока можно реализовать просто: если target = Station3, то робот доедет до Station3 (их можно пронумеровать и считать прохождение станций). Это примитивно (детекция станции у нас и так через ИК), но логика – остановиться только на нужной, остальные проезжать (как? – можно “гасить” ИК-сенсор временно). Проработаем этот алгоритм.
- **Интеграция ROS (по возможности):** параллельно (или после получения базового функционала) начнем эксперименты с ROS на RPi. Установим ROS Noetic, попробуем сделать rosnode для подписки на MQTT (можно через rosbridge, либо прямо использовать rospy + mqtt). Цель – получить, например, визуализацию траектории: можно сделать узел, который публикует `Odometry` сообщения (даже если одометрия условная, от счётчика станций). В rviz отобразим путь. Если

получится – это будет фишкой на защите, показывающей научность. Времени на полноценный SLAM мало, но мы можем реализовать **имитацию SLAM**: задать в rviz простую карту (сетки) и показать на ней точку – как будто робот локализуется. Это демонстрационно, но покажет потенциал ROS.

- **Тестирование ПО:** написать набор тестовых сценариев: 1) ручное управление – отправить последовательность команд FWD, LEFT, RIGHT, STOP, убедиться, что робот реагирует правильно (в контролируемых условиях, лучше сперва вывесить колесо, чтобы не ехал по полу без контроля). 2) Авто по линии – разметить на полу белую линию с черной окантовкой (у нас есть лента, метки), пустить робота – наблюдать, следит ли за линией. Подправить коэффициенты (DELTA поворота, скорость) для устойчивости. 3) Остановка на станции – поставить ИК-передатчик (или просто поднять робота, чтобы датчики видели черное – они должны останавливаться). 4) Препятствие – поставить коробку перед датчиком – робот должен стопориться. Мы зафиксируем логи этих тестов, при необходимости внесем коррективы.
- **Реализация клиентского интерфейса:**
- Создать веб-страницу (`index.html`) с разметкой панели управления. Подключить `mqtt.min.js` библиотеку. Настроить соединение к нашему брокеру (для тестов – `ws://192.168..:9001`).
- Написать JS-обработчики: при клике на кнопку вызывать `client.publish('agv1/cmd/move', 'FWD')` и т.п. При получении сообщения – обновлять HTML элементы (например, `document.getElementById('status').innerText = payload`).
- Проверить UI локально на компьютере (пока без робота): можно заставить его общаться с тем же Mosquitto и писать логи.
- Дополнить интерфейс графическими украшениями: иконки состояния (например, зелёный кружок – online/offline), шкала батареи (процент внутри батарейки).
- Особое внимание – отзывчивость: используем CSS Flexbox/Grid, чтобы верстка адаптировалась под разные экраны. Протестировать на ПК, планшете, телефоне.
- Добавить элементы безопасности: всплывающие подтверждения при переключении режима? (Не обязательно, но, например, при нажатии E-Stop – спросить "Вы уверены?" или сразу посылать? В критических ситуациях лучше сразу).
- Тест интерфейса: подключиться одновременно с реальным роботом и убедиться, что все действия выполняются: нажимаем “Forward” – робот едет вперед, на UI появляется “Moving forward”; отпускаем – нажимаем “Stop” – останавливается. Переключаем в Auto – робот сам поехал по линии. Это очень наглядно нужно проверить.
- **Испытания и отладка комплекса на стенде:** После того как все компоненты собраны (аппарат, ПО робота, брокер, UI), проведем целостное испытание:
- Запуск всей системы: включаем робота, он подключается к брокеру (LED можно на нем зажечь, когда connected, или UI покажет).
- Оператор с UI дает команды – робот выполняет, UI отображает обратную связь.

- Прогоняем сценарий: “Вручную довести робот до линии, переключить в AUTO, пусть проедет до следующей станции, там остановится, снова вручную объехать препятствие, опять AUTO до финиша”. Это комплексный тест всех режимов.
- Если выявятся проблемы (лаг, потеря сообщений, неправильная реакция на датчики) – вносим правки. Например, можем увеличить QoS команд до 2 (exactly once) если вдруг двусмысленность, или скорректировать алгоритм поворота, если робот виляет.
- Важно протестировать дальность связи: отойти с планшетом подальше, посмотреть, не обрывается ли. Если да, можно предусмотреть репитер Wi-Fi или ограничиться зоной.
- Провести **замер параметров**: например, задержка от нажатия кнопки до срабатывания – оценить (видео снять и проанализировать или по логам). Убедиться, что приемлемо (<0.2с).
- Проверить **автономность от облака**: отключить интернет (если используем внешний брокер) – робот должен хотя бы не разбиться: по идее, без команд он просто продолжит по линии до станции и там встанет. Связь восстановилась – он отчитался. Если что, рассмотрим fail-safe – при потере связи более N секунд в авто режиме – остановиться, ожидая команд (это можно внедрить, MQTT есть callback on_disconnect – сделаем).
- **Документирование и оформление результатов:**
- Оформить расчётно-пояснительную записку (уже делается, данный текст – основа её). Добавить недостающие разделы (экономика, БЖ – хотя для НИРС не нужно, для ВКР будет).
- Подготовить графический материал: схему архитектуры (в Visio нарисовать красиво, сейчас у нас текстом, там будет рисунок), принципиальные схемы электроники (в KiCad, возможно), диаграммы алгоритмов (UML диаграммы состояний, последовательностей – у Лукоянова были UML-диаграммы MQTT[130][72], можно по аналогии).
- Сделать фотографии модернизированного робота, скриншоты интерфейса, вставить в отчет (или вынести на слайды).
- Описать испытания, полученные характеристики (например, погрешность остановки у станции – можно измерить линейкой, сколько робот проехал от момента получения сигнала).
- Провести анализ соответствия требований: показать, что все пункты из раздела 4 выполнены или обоснованно не выполнены (например, “поддержка сценариев без перепрограммирования” – мы реализовали через отправку команд из облака, можно считать выполнено).
- Сделать выводы о преимуществах новой системы над старой: ожидается, что робот стал управляемым удаленно, гибким (легко изменить поведение через код, а не прошивку), интегрируемым в новые системы. Возможно, по итогу даже оставим его в лаборатории как учебное пособие для студентов (это отдельный плюс).
- **Подготовка к защите:** После выполнения технической части, следует подготовить демонстрацию. Планируется:

- Сделать **видео** работы робота: например, сюжет – оператор через ноутбук отдает команду, робот едет, объезжает препятствие, останавливается. Наложить поверх видео интерфейс (или снять экран тоже), чтобы было видно взаимодействие. Это видео можно будет показать комиссии (особенно если в аудитории мало места для гонок робота).
- Подготовить **презентацию** с основными результатами: структура системы (схема), фотографии, графики (можно построить график скорости или напряжения vs время, снять с телеметрии, чтобы показать, что мы можем это).
- Отметить **научную новизну**: например, сравнить с аналогами – "Вот было: проводной/ручной AGV 1999 г., стало: облачно-интегрированный робот 2025 г.", упомянуть, что применены современные технологии (MQTT, IoT Core, возможно ROS2 Gateway), что это часть тенденции Industry 4.0[3].
- Описать возможности развития: подключение системы распознавания (в инструкции Bauman FMS есть "Vision system" – мы могли бы посылать команды от системы зрения, если успеем интегрировать).
- Проговорить вопросы безопасности – мол, система тестировалась, отказов не наблюдалось, в случае сбоя есть E-Stop.

Заключение (промежуточное): На текущий момент проект *«Облачная система управления движением мобильного робота (AGV Denford)»* развивается по плану. Имеется убедительная концепция, подтвержденная как литературными источниками, так и экспериментальными пробами. Основная задача НИРС – доказать осуществимость подхода – уже решена: продемонстрирована связь робот–облако и намечены пути внедрения на реальном устройстве[121]. Впереди – интенсивная фаза практической реализации и оптимизации. Ожидается, что по завершении работы будет получен функционирующий прототип облачно-управляемого AGV. Этот прототип станет ценным как с образовательной точки зрения (платформа для новых лабораторных работ по IoT и робототехнике), так и с научной – как основа для исследований в области навигации мобильных роботов, коллективного поведения, интеграции киберфизических систем. Таким образом, проект имеет не только учебно-инженерную, но и **научную ценность**, соответствуя требованиям НИРС: в нем соединяются современные технологии и решается актуальная прикладная задача автоматизации склада на новом, облачном уровне.

---

[1] [3] [4] [5] [6] [7] [8] [9] [10] [11] [21] [37] [40] [41] [42] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [70] [71] [72] [75] [76] [77] [78] [86] [87] [90] [91] [92] [93] [94] [95] [96] [97] [98] [99] [100] [101] [102] [103] [104] [105] [108] [109] [112] [113] [115] [116] [117] [118] [121] [123] [124] [128] [129] [130] Лукоянов_Модернизация автономного мобильного робота для функционирования под контролем облачной системы.pdf

https://drive.google.com/file/d/10gUZJeWO-bRK8TbKjVq8piPhvVRoq36B

[2] Cloud robotics - Wikipedia

https://en.wikipedia.org/wiki/Cloud_robotics

[12] [79] [80] [106] [122] Ольхов (робокар).pdf

https://drive.google.com/file/d/1TiILqMRYNZn4pnuQ09s0mqPEUCXGePBV

[13] [14] [15] [16] [17] [18] [19] [20] [84] [88] [119] icas.org

https://www.icas.org/icas_archive/ICAS2020/data/papers/ICAS2020_1223_paper.pdf

[22] [23] [24] [114] [125] [126] [127] Deploy and Manage ROS Robots with AWS IoT Greengrass 2.0 and Docker | AWS Robotics Blog

https://aws.amazon.com/blogs/robotics/deploy-and-manage-ros-robots-with-aws-iot-greengrass-2-0-and-docker/

[25] [48] [49] [81] [82] [83] [85] [89] [107] [120] ВКР презентация.pdf

https://drive.google.com/file/d/1UYT9bHbp4CPTiJ-17C8wlvXSUETsl8OP

[26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [38] [39] [110] [111] AGV Manual.pdf

https://drive.google.com/file/d/1DKPu4T4FFfGhUHA431sfyLe_lFCp7pVo

[43] [44] [45] [46] [47] Инструкция по запуску системы (Редакция 6).pdf

https://drive.google.com/file/d/1hQkQ74i-xYIvmDPhqmMudj4G6ub2ALGl

[68] [69] описание подключений.pdf

https://drive.google.com/file/d/1K3kK5PrIUNGFqK14DtF8Twg6dN-4FiBT

[73] [74] Половников_Разр.прогр.части сист.упр.ПР с использ.языка MicroPython на базе аппар.платф.ESP32.pdf

https://drive.google.com/file/d/1fTi-uK23qrwRJcEGCj2GXambXwH2_qb3